



Rensselaer

Capstone Design Project Final Design Report

Aircraft Robotic Assembly

Project Client
Boeing

Fall 2025
December 2025

Prepared by

Darwin Capani, Electrical Engineering
Talon Carsto, Electrical Engineering
Anderson Corriero, Electrical Engineering
Jeremia Silaban, Mechanical Engineering
Jay Tang, Computer Systems Engineering
Brody Waters, Electrical Engineering
Peijing Xu, Computer Systems Engineering + Computer Science

Project Engineer Kannathal Natarajan - CORE Engineering
Chief Engineer Junichi Kanai – ECSE Department

Table of Contents

Table of Contents	2
Table of Figures	4
List of Tables	5
Glossary	6
Acknowledgements	8
Executive Summary	9
1 Introduction	11
2 Project Overview	12
2.1 Project Statement	12
2.2 Project Objectives (Long-Term)	12
2.3 Semester Objectives / Deliverables (Short Term)	13
3 Engineering Definition of Problem	14
4 System Concept Development	19
4.1 System Upgrade	19
4.2 Visual Feedback	21
4.3 Ramp Navigation	25
4.4 Cable Detection	30
4.5 Gripper Re-design	37
4.6 Pathfinding Algorithm	40
4.7 Safety Features	45
5 System Proposed Design	48
5.1 System Upgrade Final Design	48
5.2 Visual Feedback Final Design	49
5.3 Ramp Navigation Final Design	52
5.4 Cable Detection Final Design	56
5.5 Gripper Re-design Final Design	58
5.6 Pathfinding Algorithm Final Design	60
5.7 Safety Features Final Design	63
6 System Evaluation	66
6.1 System Upgrade	68
6.2 Visual Feedback	68
6.3 Ramp Navigation	71
6.4 Cable Detection	72
6.5 Gripper Re-design	76
6.6 Pathfinding Algorithm	76
6.7 Safety Features	76

7 Significant Accomplishments and Recommendations	77
7.1 System Upgrade	77
7.2 Visual Feedback	78
7.3 Ramp Navigation	79
7.4 Cable Detection	83
7.5 Gripper Re-design	84
7.6 Pathfinding Algorithm	84
7.7 Safety Features	85
8 Conclusion	87
9. References	89
Appendix A: Standards	91
Appendix B: Bill of Materials	93
Main BOM	93
Fall 2025 BOM	94
Appendix C: Ethical and Professional Responsibilities	95

Table of Figures

Figure 1: Raspberry Pi LED Connection	23
Figure 2: Counterweight Concept	26
Figure 3: Compass Concept.....	27
Figure 4: Track Concept.....	28
Figure 5: Intel RealSense RGB-D camera mounted on the robotic platform.....	30
Figure 6: Processing flow of the cable detection pipeline.....	32
Figure 7: Example runtime outputs.....	33
Figure 8: A 3D printed model of a fractal vise [8]	39
Figure 9: Reactive Pathfinding	40
Figure 10: Deliberative Pathfinding.....	41
Figure 11: Return home feature	43
Figure 12: Aluminum Angle CAD Drawing	49
Figure 13: Angle Installation on Arm Base.....	50
Figure 14: Flow Chart for Final Visual Feedback Design	51
Figure 15: LED Installation on Robot.....	52
Figure 16: Compass mount CAD model	53
Figure 17: Compass mount installs	54
Figure 18: Compass chip in mount	55
Figure 19: Flowchart for code logic.....	56
Figure 20: Cable detection camera position and terminal results	58
Figure 21: New Gripper CAD design	59
Figure 22: 3D printed gripper mounted on robotic arm.....	59
Figure 23: A* algorithm implementation.....	61
Figure 24: Autonomous Mode.....	62
Figure 25: Battery housing with emergency stop button and rear ultrasonic sensor	64
Figure 26: Ultrasonic sensor mounted to the front left of the robot	65
Figure 27: Compass Installation	72
Figure 28: Cable detection functionality test results.....	74
Figure 29: Cable detection node level test results	75
Figure 30: Robot COM top view	80
Figure 31: Robot COM front view.....	81
Figure 32: Robot COM side view	82

List of Tables

Table 1: High Priority Needs and Requirements	14
Table 2: Project Use Cases	16
Table 3: Project User Stories	17
Table 4: System Upgrade Concept Selection Matrix.....	20
Table 5: Visual Feedback Concept Selection Matrix.....	24
Table 6: Ramp Navigation Concept Selection Matrix	28
Table 7: Cable Detection Concept Selection Matrix	36
Table 8: Gripper Re-design Concept Selection Matrix.....	39
Table 9: Pathfinding Algorithm Concept Selection Matrix	44
Table 10: Safety Features Concept Selection Matrix.....	45
Table 11: Individual Tech Demo	66
Table 12: Color/Pattern Outputs for Robot States	69
Table 13: Robot State Input Bits.....	70
Table 14: Standards Table	91
Table 15: Main BOM.....	93
Table 16: Fall 2025 BOM.....	94
Table 17: Team Ethical and Professional Responsibilities	95

Glossary

Term	Definition	Description
PS4 Controller	Play Station 4 Controller	A controller that is connected to the robot via Bluetooth and its inputs are mapped to control the base and arm position
CMPS12	Electronic compass	The CMPS12 is the name of the compass module that will be used throughout the project, specifically in the ramp navigation section.
E-Stop	Emergency Stop	User input designed to stop all current processes (software and hardware) in the event of an emergency.
GPIO	General Purpose Input/Output	General purpose pins/ ports that allows for controlling motors, servos, and receiving external data such as sensor data and user input
GUI	Graphical User Interface	A graphical (most times windows on a screen) interface that allows the user to interact with the system, such as input values or to read system status/ outputs.
HMI	Human-Machine Interface	A layer of software and hardware that allows communication between the user and the machine, often utilizing GUIs and various feedback mechanisms such as speakers and screens
IEC	International Electrotechnical Commission	An organization which prepares and publishes international standards for all electrical, electronic, and related technologies. Certain IEC standards [2] concerning electrical and robot safety are used as reference in our design process.

Term	Definition	Description
ISO	International Organization for Standardization	An independent, non-governmental, international standard development organization. Defines safety and electrical standards [3], many of which are used as reference for informed design of our project.
LiDAR	Light Detection and Ranging	A sensor that uses laser pulses to measure the distance from a point. This is used in the project for obstacle detection so they can be avoided.
Pi	Raspberry Pi 4, 5	Light weight computer running Linux based operating system. The center processing unit of the robot serves as the “brain” for most higher-level processes.
ROS / ROS2	Robot Operating System	This is a set of software libraries and tools to control a robot. It allows for integration of a variety of hardware as “nodes”, and common data streams as “topics”. Our robot is running a version of ROS on Ubuntu 22.04 LTS
Vosk	Vosk	An open-source python voice recognition library with modular data models allowing for multiple levels of accuracy and processing needs.
Heuristics	Heuristics	A method that is designed to find a satisfactory solution in a reasonable amount of time.
SLAM	Simultaneous Localization and Mapping	The process of creating a map using a robot that navigates an environment while using the map it generates.

Acknowledgements

We would like to extend our sincere appreciation to everyone who contributed to the success to our project.

We are especially grateful to our RPI Sponsors. Santiago Paternain offered valuable insight into the demonstration environment and provided essential materials that supported the development of our product design enhancement subsystem. We also thank Glenn Saunders for his continued guidance throughout the project.

We acknowledge our Boeing stakeholder, James Brogan, for his encouragement and support.

We are deeply appreciative of the direction and expertise provided by our Project Engineer, Kannathal Natarajan, and our Chief Engineer, Junichi Kanai, both of whom played a key role in shaping our project's progress and outcomes.

Finally, we thank the RPI Design Lab for supplying the tools, resources, and workspace that made this project possible.

RPI Sponsor: Santiago Paternain

RPI Sponsor: Glenn Saunders

Boeing Stakeholder: James Brogan

Project Engineer: Kannathal Natarajan

Chief Engineer: Junichi Kanai

RPI Design Lab

Executive Summary

The Aircraft Robotic Assembly project, sponsored by Boeing, aims to lessen the physical challenges associated with the installation of heavy cable bundles in aircraft production by developing an autonomous robotic assistant to assist technicians with the transport and installation of cables. To reach this goal, the Fall 25 team works to enhance the autonomous functionality of the robot by improving cable detection and autonomous navigation so the robot will be better at navigating to cables by itself. Alongside that, the Fall 25 team enhanced safety by adding more sensors cover blind spots and a visual feedback subsystem so crew members will have an easy way of seeing the current status of the robot. These efforts extend earlier progress to further help refine the robot's performance and operator interactions, preparing the system for the demands of the manufacturing atmosphere it will be in.

Key advancements were made over this semester, such as upgrading hardware, creating a visual feedback system, installing a sensor for the robot to detect tilt, improving safety features, re-designing the gripper, and improving the autonomous mode navigation and cable detection software.

Software advancements were made to autonomous mode operations, which is now way more flexible in handling obstacles, being able to detect faster obstacles and some non-stationary obstacles as well. The new autonomous mode can accurately go from point A to point B with better accuracy and a higher sense of security as the robot will no longer be bothered by faulty

algorithms and dynamic obstacles. Improvements are also made to cable detection software which utilizes python packages to more accurately detect where the cable is. Lastly, the navigation upgrade gave the robot a compass, which it will then use to detect when the robot is tilted upwards and can adjust itself so that its center of mass is at the perfect point where the robot can safely go up ramps as the current model will always slip.

Hardware advancements include an upgrade from the Raspberry Pi 4 to the Raspberry Pi 5, which boasts more computational power and better connectivity. Another upgrade made was the visual feedback system, where given a binary code, a strip of LEDs would light up and indicate what mode the robot is currently in, like purple for autonomous mode, red for emergency stop, and so on. This add-on improves the quality of life for debugging purposes and serves as a nice visual for presentation. Another add feature is the safety mechanism, where the emergency shutdown will be more robust and easier to access than before, now sensors will also be initiated to detect if obstacles are too close to the robot, residing in the blind spot of the LiDAR used by autonomous mode. Finally, a total redesign of the grippers is to ensure an increase in the reliability of the arm grabbing a cable, by increasing the grip force and making sure there is no slippage.

1 Introduction

Modern aircraft contain extensive wiring systems that connect sensors, communication networks, control modules, and entertainment units throughout the structure. These cable bundles are often heavy and difficult to maneuver, making installation physically demanding and potentially hazardous to technicians. To address these challenges, the Aircraft Robotic Assembly team has been commissioned by Boeing to design a robotic system (robot) that will assist human technicians in the installation of cables. The robot will be able to pick up bundles of cables, navigate around a workshop while holding the cables, and position the cables where needed so that human technicians can install them with ease. The aim is to have these tasks done autonomously, with the option of tele-operation as needed for more difficult maneuvers.

Previous teams have established a solid foundation for the Aircraft Robotic Assembly project. The Spring 24 team ported the robot over to the ROS2 framework currently used on the robot. The Spring 25 team created the packages for the robot's autonomous mode and laid out the groundwork for a voice control feature [1]. The current team, the Fall 25 team, aims to build upon the progress made by the previous teams by fixing edge case errors and improving the autonomous mode functionality. Key objectives include adding visual feedback responses to robot commands, enhancing the claw's performance, and optimizing the robot's navigation and camera detection software.

2 Project Overview

2.1 *Project Statement*

The Aircraft Robotic Assembly project focuses on the development of a robotic system capable of assisting technicians with the transportation and installation of aircraft cable bundles. The goal of the project is to reduce the ergonomic strain and safety risks faced by technicians while improving the efficiency of the cable installation process. Building on the foundation of the previous teams, the Fall 2025 team aims to further refine the robot's performance by improving autonomous mode software, improving the gripper design, creating a visual feedback system, and ensuring full compliance with industrial safety standards. These efforts will culminate in a fully functional robotic system capable of operating reliably in a simulated manufacturing environment that mirrors Boeing's real-world assembly process.

2.2 *Project Objectives (Long-Term)*

The long-term version of the Aircraft Robotic Assembly project is to create a robust, industrial-grade robotic assistant that can be used in Boeing's workforce to assist technicians in cable routing and assembly. The system should be able to perform repetitive tasks both in tele-op and autonomous modes while maintaining high precision and safety [1]. Having the system collaborate efficiently with human operators aboard aircraft is a key factor for long-term objectives. One key milestone for the long-term objectives is achieving a fully autonomous operation mode, where the robot will be able to navigate to a cable, pick it up, and go to the place of installation without any external input. This milestone, along with others, satisfies the needs of Boeing because human technicians will be able to focus on higher precision installation work without the more physically demanding transport and routing work.

In addition to this milestone, the robot should maintain compliance to all applicable safety standards, which include software, mechanical, electrical, and industrial standards. This includes a fully implemented e-stop system, with diagnostic functions to determine the reason for stop and safe power-down procedures that minimize the risk of injury from erratic movements when power is lost. Other long-term objectives include creating a refined and durable physical design, which will help the ease of upgrading in the future. Ultimately, these approaches will provide a seamless robot design that meets the client's requirements through an improved autonomous system and aesthetic design.

2.3 Semester Objectives / Deliverables (Short Term)

The Fall 2025 team will attempt to accomplish the following objectives:

1. Upgrading the processor to a Raspberry Pi 5
2. Improving the autonomous navigation capabilities with more frequent LiDAR data feedback loops and a better pathfinding algorithm,
3. Implementing a return to home feature that returns the robot to its original position after successful cable pickup
4. Improving the ability to scale ramps without drifting
5. Refining gripper design and force curve that can more reliably pick up cables of varying lengths and weights
6. Implementing a visual feedback system to communicate the current operational status of the robot (i.e. ready to receive commands, searching for cable, etc.)
7. Adding the following safety features:
 - Ultrasonic sensors that constantly monitor blind spots
 - An e-stop button that cuts power to the wheels

3 Engineering Definition of Problem

The high priority needs and requirements for Fall 2025 is listed below, and serve as a comprehensive reference, detailing the critical factors that must be addressed to ensure effective implementation and optimal outcomes. Each entry highlights specific needs, their associated importance, and outlines the necessary requirements to fulfill these demands.

Table 1: High Priority Needs and Requirements

#	Need	Priority	Req #	Requirement Description	Value(s) / Range	Units
1	Customer needs robot's autonomous function improved	High	1.1	The robot shall detect and navigate ramps in the environment.	≤ 15	degrees (ramp angle)
1		High	1.2	Learn and map the working environment using sensor data	≥ 95	% (mapping coverage accuracy)
1		High	1.3	Help improve localization accuracy using LiDAR and IMU sensor fusion	$\leq 5-10$	cm (position error)
2	Customer needs a more reliable gripper	Lower	2.1	The gripper should reliably pick up and hold cables of varying sizes and weights	≤ 1	kg
2		High	2.2	The gripper should not grip tightly enough to break cables	≤ 100	psi
3	Customer needs the robot to be tested in a similar environment to deployment	Medium	3.1	Test area should be a suitable size	>10	m ²
3		Medium	3.1	Test area should contain a ramp	≤ 15	degrees (ramp angle)

#	Need	Priority	Req #	Requirement Description	Value(s) / Range	Units
4	The customer needs the robot to be safe	High	4.1	Robot should have a power cutoff switch	≤ 1	second (Turn-off time)
4		Medium	4.2	Robot should avoid obstacles through sensor-based detection.	≥ 95	% accuracy
5	Customer needs robot to provide visual feedback on command execution	High	5.1	Must provide clear and immediate feedback to the user on robot's current state	≤ 100	ms (Response time)
5		High	5.2	Must ensure output is visible/loud enough from a certain distance from the robot.	≥ 2	meters (Distance from robot)
5		Medium	5.3	Must operate within robot's available power supply.	≤ 5	Volts
5		High	5.4	Must have multiple unambiguous states (Distinct)	≥ 6	distinct states
6	Voice commands	High	6.1	Robot should be able to recognize a suitable number of commands	10	commands
6		High	6.2	Robot should hear commands well in loud environments	< 70	dB
7	Give user notifications	Medium	7.1	Warn operator once battery decreases to a certain percentage	20	%
7		High	7.2	Inform operator when errors occur	< 10	seconds

Table 1 shows the overview of the team's 7 highest priority needs and the requirements for each of them. The customer needs the robot's autonomous function to be improved upon, meaning it should navigate and detect ramps in the environment, and can map the working

environment using its sensor data. The customer also needs a more reliable gripper for cable assembly, meaning it should reliably pick cables up, despite their size, of 1 kg or below. The working environment should be set up to demonstrate the industrial working environment Boeing has. Ramps and obstacles should be placed inside the test area. Additionally, safety standards should be in place so the robot is safe to operate in any case. The customer will need the robot to provide clear and immediate feedback to the user on command execution, which may be from voice commands or pathfinding. The system needs to warn the operator if the battery is at a lower level as well.

The project uses cases that illustrate how the outlined requirements and needs will be applied in real-world scenarios. Each use case provides a detailed description of our goal at the end of the semester, showcasing the practical applications of our solutions.

Table 2: Project Use Cases

Title	Primary User	Goal
Voice commands	Operator	Use voice commands to operate the robot
Grab Cable	Operator	Be able to grab various sizes of cable
Incline climbing	Operator	The robot should be able to climb slight to moderate inclines
Accurate Movement	Operator	The robot moves in the direction desired
Obstacle Avoidance	Robot	The robot should be able to avoid obstacles with precision
Obstacle Detection	Robot	The robot should use LiDAR information to detect obstacles

Title	Primary User	Goal
Visual Feedback	Robot	The robot must respond to the user using visual feedback for actions completed

From Table 2, the project use cases can be divided into two main categories: Receiving inputs and executing the inputs. The inputs focused on by the development team are: voice commands and LiDAR/Camera information. The outputs focused on are: grabbing cables, moving without drift, and visual feedback that shows the robot is indeed performing the given input.

Project user stories encapsulate the perspectives and needs of the end-users who will interact with our solution. Each user story is structured to describe a specific functionality from the user's viewpoint, detailing what they want to accomplish and the benefit of that capability.

Table 3: Project User Stories

As a [type of user]	I need to [perform some tasks]	So that I can [achieve some goals]
As a Boeing Engineer	I want the robot to function as intended with no issues.	So that human casualties can be prevented or minimized during airplane construction
As a technician	I want the robot to autonomously detect and navigate ramps	So that I can use it in multi-level assembly areas without manual adjustments.
As a manager	I want the robot to be compliant with safety standards	So it can be used on factory floors
As a aircraft technician	I want the robot to respond to voice commands that is no softer than a normal talking voice.	So it is easy and intuitive to work with
As a Boeing Engineer	I want the robot to be easy to use	So I don't have to waste too much time with training

As a [type of user]	I need to [perform some tasks]	So that I can [achieve some goals]
As Boeing Floor Manager	I want the robot to increase worker output / productivity	So that we can produce more planes with the same level of safety and quality

Table 3 describes the user stories for the Aircraft Robotic Assembly project. The team assesses the different types of operators that could use this robotic assembly. This includes a Boeing engineer, a technician on an aircraft, a manager, as well as a Boeing floor manager. These users need the robot to comply with safety standards, operate without internal or external issues, listen and respond to voice commands, as well as increase worker output and productivity. These needs are necessary for the users so that they can be used for ease of work, and any casualties can be prevented.

4 System Concept Development

4.1 System Upgrade

System Upgrade concepts were selected to meet the team's requirements for a higher power processor that kept within the power limits. The selected processor would need to be able to support ROS2 Humble, the underlying software library used for the robot, as well as have ample hardware ports for the necessary connections to other hardware.

Concept 1: Jetson Nano

The Jetson Nano is a single-board computer developed by Nvidia for use in generative AI. It has the highest compute power out of all the units and has equivalent electrical power draw to the currently installed Raspberry Pi 4. Previous groups have experimented with using the Jetson Nano. However, Nvidia has a poor track record of support for Linux, having already ended support at Ubuntu 18.04. This means that docker will have to be used to virtualize a ROS2 Humble compatible Linux version, which already has proven software compatibility issues with the Interbotix VX-300 arm being used. Upgrading the arm is outside the scope of the current semester's objectives, so the Jetson Nano was eliminated from consideration.

Concept 2: Raspberry Pi 4 Cluster

A Pi cluster is a group of multiple Pis connected together with a data-transfer capable wire, usually ethernet. From this connection, the Pi's will have access to each others' computing capabilities. While this doesn't increase the Pi's speed for single-threaded instructions, this can greatly improve performance in parallelized tasks by doubling the amount of processors.

However, the electrical power draw increases by 15W every single time a Pi is added, which means that the electrical system will need to be upgraded to power the Pi cluster. Upgrading the electrical system is also outside the scope of the current semester's objectives, so the Pi cluster was eliminated from consideration.

Concept 3: Raspberry Pi 5

The Raspberry Pi 5 is the successor to the Raspberry Pi 4, offering nearly triple the performance of the Raspberry Pi 4 with only a 10W increase in power draw at the top end. This 10W increase can be mitigated by capping the wattage of the processor to 15W to keep in line with the current power limits. At this power limit, the Raspberry Pi 5 will only lose ~20% performance, still 2.4x the performance of the Raspberry Pi 4. The hardware design for attachments on the Pi 5 is the same as the Pi 4, meaning the USB HAT+ attachment will work without any adjustments, as well as all the other peripherals. Because of how easily compatible the Pi 5 with the existing project parameters, the Pi 5 was ultimately the concept of choice.

The concept selection matrix for the system upgrade subsystem is below, with the requirements that the team is trying to achieve and the metrics measured from each concept.

Table 4: System Upgrade Concept Selection Matrix

#	Requirement	Metric / Unit of Measure	Jetson Nano value	Raspberry Pi 5 value	Pi 4 Cluster value
5	Power Usage <15W	Wattage	15-25	15-27W	30-36W
1	Compatibility with existing software/hardware	Yes/No	No	Yes	Yes
1	Computational Power in FP32	GFLOPs	235.8	31.4	13.5
5	External device support via GPIO	mAmps/V	2/5	16/5	16/5

4.2 Visual Feedback

Concepts within the Visual Feedback subsystem were selected to meet the team's needs and requirements for testing. Each concept addressed the problem of limited feedback from a different sensory perspective which is visual, textual, and auditory. Their feasibility was analyzed using criteria such as visibility, response time, power draw, and ease of integration with the existing system.

WS2811 LED Strip

The WS2811 LED Strip was originally selected as the preferred concept. This addressable RGB LED system offers multiple colors and pattern capabilities that can be programmed to represent different robot states. It is highly visible and can be mounted around the robot's base to provide a 360-degree field of visibility for operators. For testing purposes, the LED strip can instantly communicate operational changes, such as transitioning from idle to being in motion, from tele-op or autonomous mode. For testing purposes, the LED strip can instantly communicate operational changes, such as transitioning from idle to being in motion, from tele-op or autonomous mode. It will integrate directly with the Raspberry Pi using a single GPIO pin for data and is powered by the existing 12-volt power supply on the robot. A variable resistor will most likely be used to limit the current draw to make sure the Raspberry Pi doesn't break. The LED strip is connected in pairs of three IC chips (LEDs) in line with each section. When programming this component, one "rail" represents three LEDs, meaning that having only one of the three LEDs being active is not possible with this model.

WS2812 LED Strip

This LED strip model, which is an updated model of the WS2811 version, was ultimately decided to be the concept in the final design. This LED strip was manufactured by the same company that created the WS2811 model, Adafruit Industries. This component offers the same capabilities and uses the RGB color code for implementation, however runs on a 5-volt supply instead of 12 12-volt supply. This allows the component to be powered and controlled on the Raspberry Pi, which decreases the electrical risks the LEDs may have. This LED strip uses 5-volt power and requires 5 volts for its logic controller, meaning a variable resistor is not required, and the component can be connected straight to the power source, data input pin, and ground. Each LED on the strip represents one “rail”, so each light can be controlled independently instead of being required to control three at the same time. The figure below shows the connection with the Raspberry Pi.

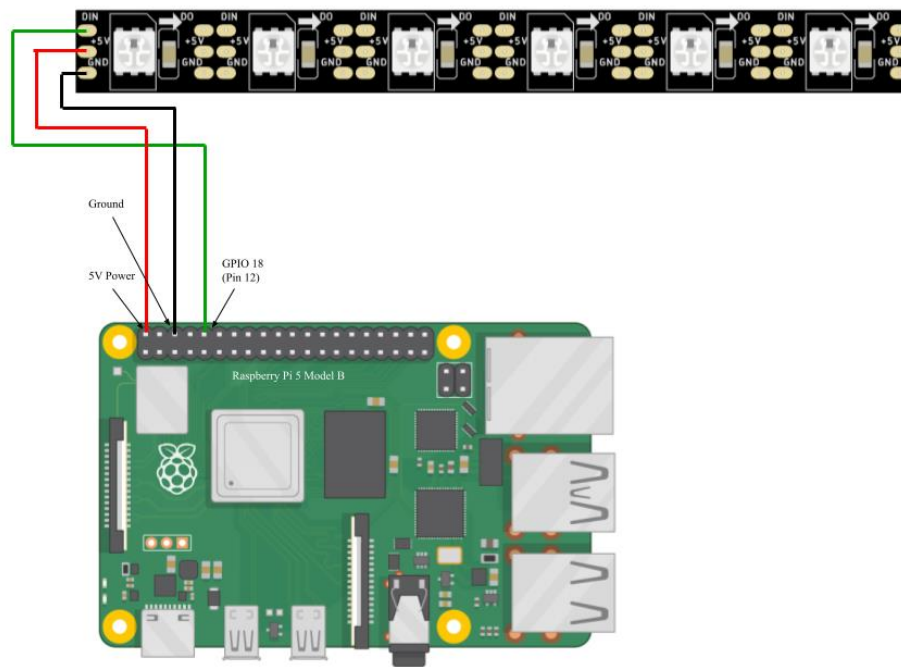


Figure 1: Raspberry Pi LED Connection

LCD I2C Screen

The LCD Screen, also known as the Liquid Crystal Display, was considered as the Visual Feedback subsystem's choice of feedback output. The small display module can print short text strings such as "Cable Detected" or "Moving to Home Position", providing detailed information beyond what LED colors alone can convey. However, its narrow viewing angle and limited readability under bright lighting conditions posed drawbacks for use in a factory setting. The LCD screen would also require more frequent software updates to format messages appropriately, which adds unnecessary complexity.

Monk Makes Speaker

An auditory feedback device is beneficial in an industrial environment due to its ability to alert and catch the attention of operators in a louder environment. This simple tone generator can output short beeps or warning tones corresponding to robot events, such as obstacle detection. This output device would greatly impact the Safety Features subsystem, because it will alert operators if the emergency stop button is activated. While it effectively supplements visual feedback, it was not chosen for final implementation because it provides only one distinct state to the operator, while the needs and requirements require the system to provide multiple distinct states. The speaker may also contribute to unnecessary noise during testing.

Below, the concept selection matrix for the visual feedback subsystem is located and shows which of the concepts, defined previously, is most suitable for meeting the requirements of Fall 2025 Aircraft Robotic Assembly.

Table 5: Visual Feedback Concept Selection Matrix

#	Requirements	Metric/ Unit of Measure	WS2811 LED Strip	Monks Makes Speaker	LCD I2C Display	WS2812b LED Strip
5.1	Must provide immediate feedback on the robot's current state	≤ 100 (ms)	Can achieve the targeted unit of measure in under 50 milliseconds.	Can achieve the targeted unit of measure in under 50 milliseconds.	Does not reach the targeted unit of measure, takes about 150 milliseconds.	Can achieve the targeted unit of measure in under 50 milliseconds.
5.2	Must ensure output can be acknowledged at a certain distance from the robot.	≥ 2 (meters from robot)	It can be seen from 3-4 meters away.	This can be heard from about 2-3 meters away.	This does not reach the targeted unit of measure; it can be seen only 1-	It can be seen from 3-4 meters away.

#	Requirements	Metric/ Unit of Measure	WS2811 LED Strip	Monks Makes Speaker	LCD I2C Display	WS2812b LED Strip
					1.5 meters away.	
5.3	Must operate within the robot's available power supply.	≤ 5 (volts)	Requires 12 volts to operate.	Requires 5 volts to operate.	Requires 5 volts to operate.	Requires 5 volts to operate.
5.4	Must have multiple unambiguous states (Distinct)	≥ 6 (Distinct states)	It has more than six unambiguous states.	Only has one unambiguous state.	It has more than six unambiguous states.	It has more than six unambiguous states.
7.2	Inform the operator when errors occur.	< 10 (seconds)	Meets the targeted unit of measure.	Meets the targeted unit of measure.	Meets the targeted unit of measure.	Meets the targeted unit of measure.

4.3 Ramp Navigation

Concepts within the Ramp Navigation subsystem were selected to meet the team's needs and requirements for testing. All concepts generated are for the mitigation/elimination of drifting on inclines. Their feasibility was analyzed using criteria such as difficulty of implementation, time to complete, and price.

One of the potential solutions is weight balancing using a counterweight. In the robot right now, there are electronics asymmetrically placed causing the robot to have an offset center of mass. This is one of the reasons that the robot drifts on inclined surfaces. To counteract this drift, the center of mass must first be found through simulation and physical measurements. Once the center of mass is found, a counterbalance weight can be manufactured and implemented. An alternative to a manufactured weight is using the weight and maneuverability of the arm to make

a dynamic counterweight that can adapt to different situations. Why this is a novel solution, it is a bit too much work to realistically achieve for a concept that may not even fully address the issue at hand.

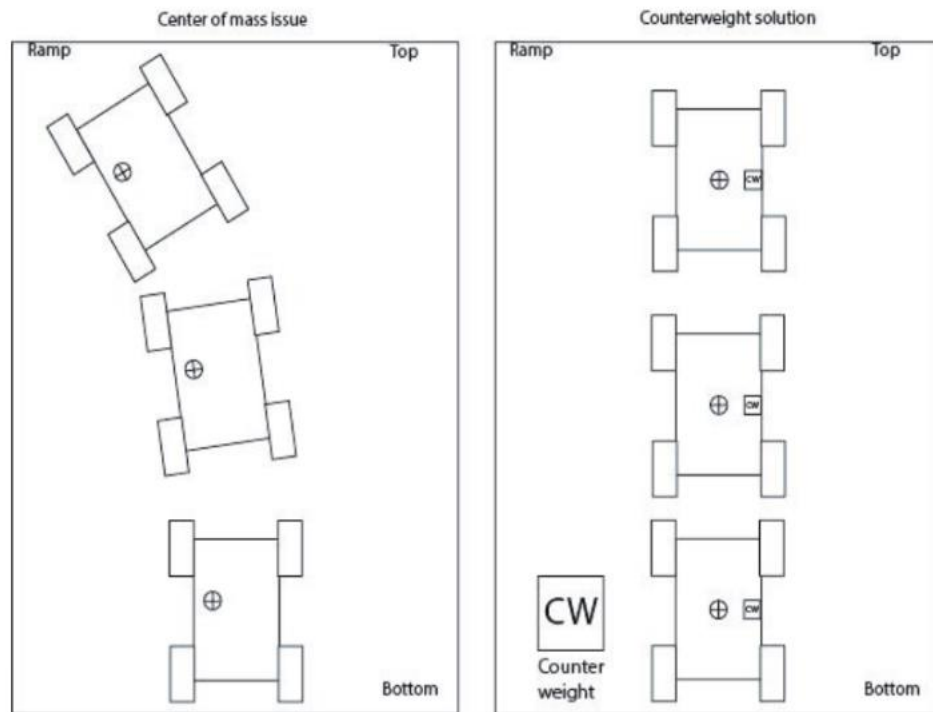


Figure 2: Counterweight Concept

A second potential solution to the drifting issue is to implement a corrective compass. This compass would detect deviations from the expected route and correct the drift experienced. While implementing a compass in the existing system will be challenging, the benefits of its successful implementation will mean that the robot should be able to correct drift experienced anywhere, not just on ramps. This will aid not only in the robot's ramp navigation, but also its ability to navigate a larger testing area while keeping its heading consistent.

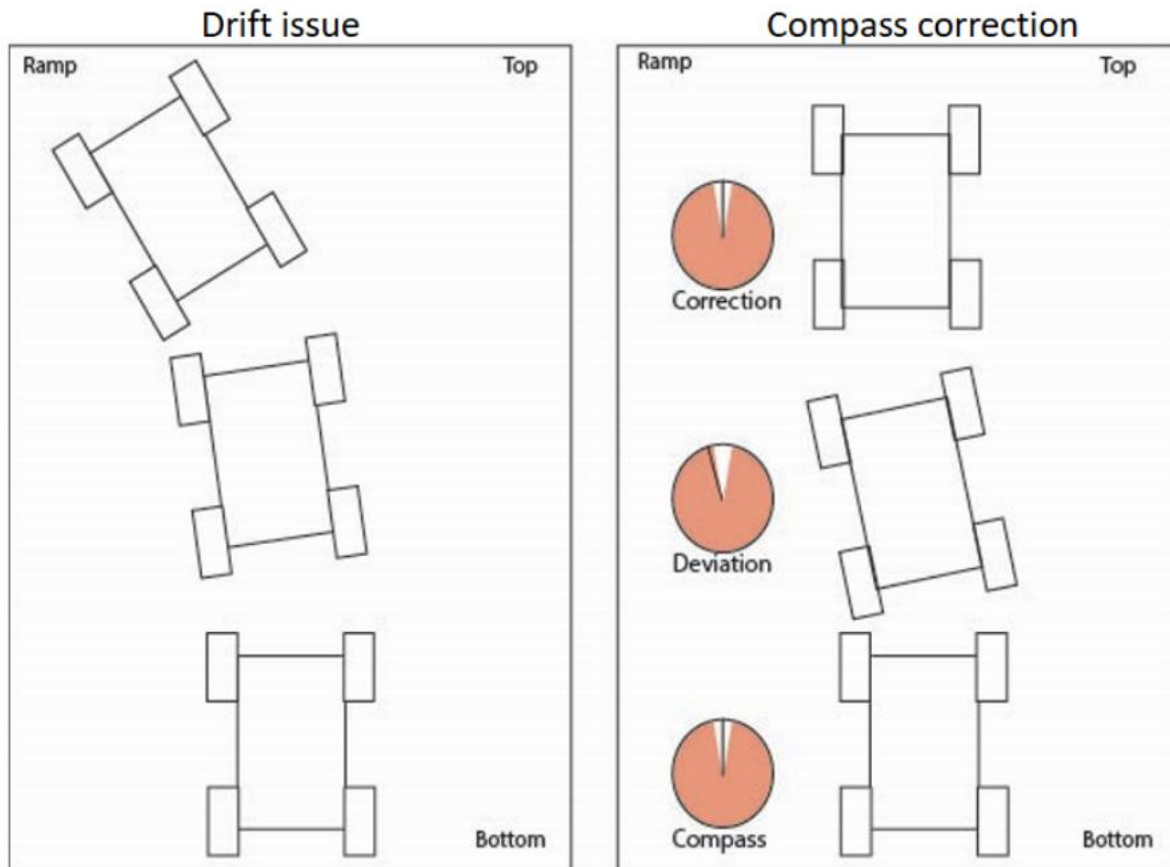


Figure 3: Compass Concept

The third potential solution to correct the drifting on ramps is to switch to tracks. While tracks would be a very large change to the overall structure and function of the robot, it would also be very effective at reducing the drift experienced on ramps. Tracks have a much better grip than the current omni wheels and would result in a lower and more stable center of mass. The greater traction and lower center of mass would both be very effective at reducing drift experienced anywhere on the testing course.

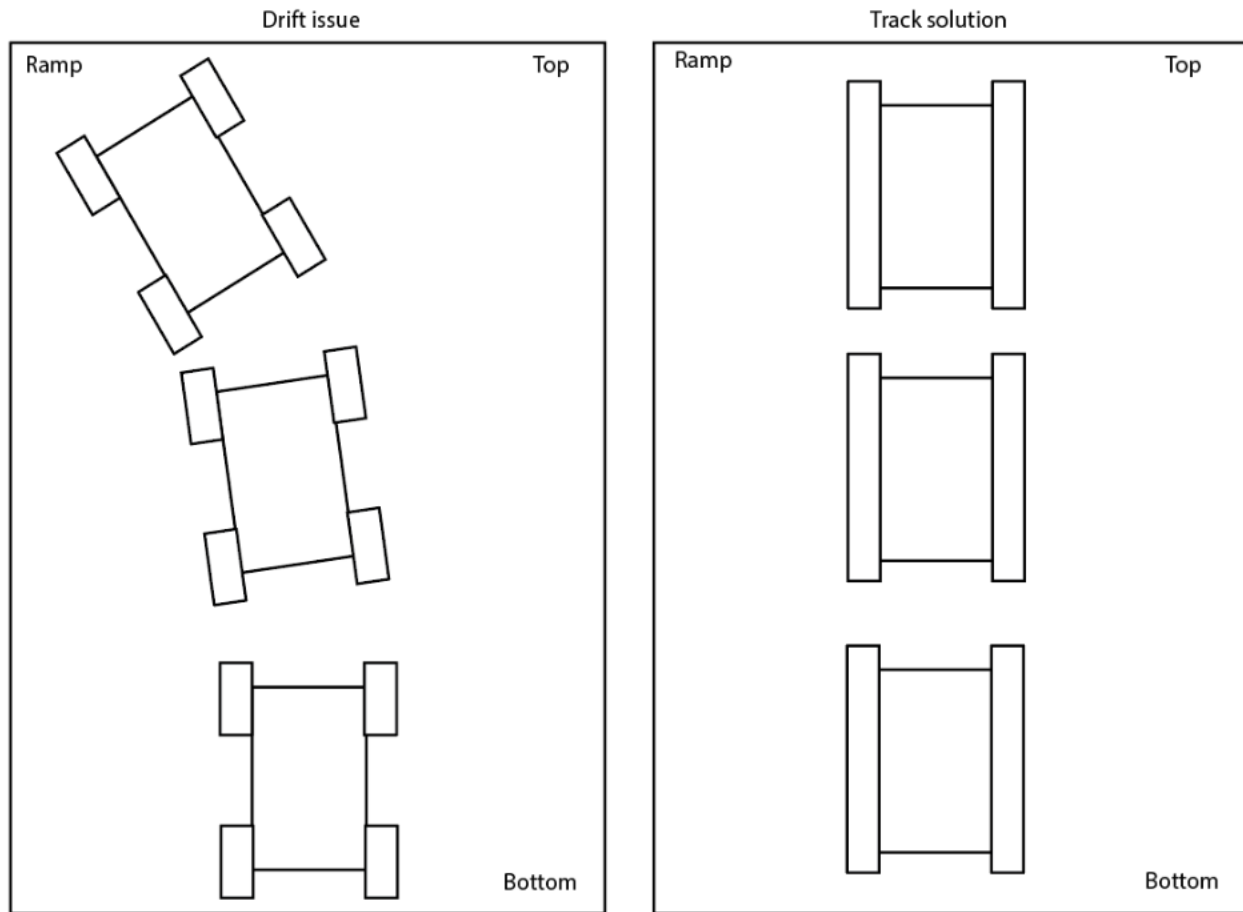


Figure 4: Track Concept

To better compare these options against each other, these concepts were put into a table of the project requirements.

Table 6: Ramp Navigation Concept Selection Matrix

Requirement	Description	Metric/Unit of Measure	Counterbalance	Compass	Treads
Objective of Subsystem	The robot shall detect and navigate ramps in the environment	Will the concept solve the problem?	Not on its own	Yes	Yes

Requirement	Description	Metric/Unit of Measure	Counterbalance	Compass	Treads
Ease of implementation	Does the solution require integration with previous subsystems?	Yes/No	No	Yes	Yes
Timeline	How quickly can the new subsystem be integrated with the rest of the robot?	Time	Minimal, weight analysis and counterweight design	Intermediate, new parts and integration required	Advanced, large-scale disassembly and reworking of the robot
price	How much will the new subsystem cost to add?	USD	Inexpensive	Moderately expensive	Very expensive
Selection			X	X	

Ultimately, we decided to go with the counterweight and the compass. This is because the compass and counterweight provided the most economical solutions. While the counterweight is unlikely to fix the drifting issue on its own, its ease of implementation makes it worth the benefits it can provide. For the compass, its universal corrective applications make it an ideal solution despite the difficulties that installing new hardware will bring. The tracks didn't make the cut because it would require a full disassembly of the current wheel system and new drive code to control the tracks. Though it may have been effective at preventing drift, the tracks were simply not worth the effort required to implement them.

4.4 Cable Detection

The concepts selected for the cable detection were chosen based on user needs, cost, and robustness. This section documents the technical implementation of the cable detection software package developed for the capstone system. The package performs real-time detection of a cable in the camera view, visualizes the detection as a green line overlay, and computes the cable's 3D position (X, Y, Z) using an Intel RealSense RGB-D depth camera integrated with ROS 2. The software is designed as a modular pipeline that subscribes to synchronized color and aligned depth topics, isolates the cable in the image, samples depth at cable pixels, and converts the resulting pixel locations into camera-frame coordinates suitable for downstream autonomy modules.



Figure 5: Intel RealSense RGB-D camera mounted on the robotic platform

During development, the cable detection package was upgraded from an initial proof of concept into a ready ROS 2 component with clear interfaces, stable runtime behavior, and outputs that are directly usable by other software on the robot. The upgrades focused on reliable

RGB and depth data, a processing pipeline for detection and 3D localization, visualization for rapid validation, and ROS 2 topic interfaces for integration with autonomy stacks.

ROS 2 Integration with RealSense

A major upgrade was the full integration of the Intel RealSense RGB-D camera into the ROS 2 computation graph. The RealSense driver node publishes the camera's color stream and an aligned depth image that is registered to the color frame. Using aligned depth is essential because the cable detector identifies cable pixels in the color image, and the depth sampled for those same pixels must correspond to the same physical points in the scene.

The package is structured so that the RealSense node and the cable detection node can be launched together or independently. This separation improves debugging and makes the cable detector reusable across platforms.

Detection-Localization Processing Pipeline

The package was upgraded into a modular pipeline, where each stage has a specific responsibility and explicit inputs and outputs. Figure 3 shows the data flow: camera acquisition, ROS 2 publication of color and depth, cable detection, depth sampling at detected cable pixels, and conversion to 3D coordinates.

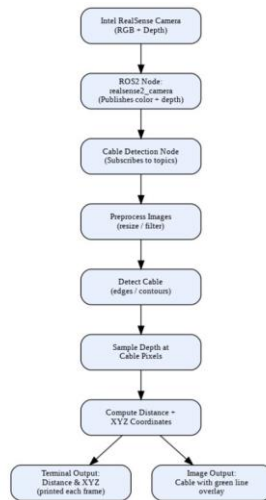


Figure 6: Processing flow of the cable detection pipeline

Within the cable detection node, image processing is performed to stabilize detection quality across frames. Processing steps commonly include resizing, noise reduction, and filtering operations that improve the signal-to-noise ratio before contour- or edge-based detection. The detection stage then isolates the cable geometry from the background and produces a set of cable pixels (or a fitted curve/line representation) used for downstream computation.

After detection, the node samples the aligned depth image at the locations of the detected cable pixels. Sampling depth at the cable itself (rather than at an arbitrary point in the frame) is a key feature that enables accurate 3D localization. The sampled depth values are then combined with the camera intrinsics to compute the cable's X, Y, and Z coordinates in the camera frame, in meters. These coordinates can be published as point data for integration with navigation, manipulation, or inspection behaviors.

Real-Time Visualization and Operator Feedback

To support debugging and demonstrations, the package adds visualization outputs that make the system behavior immediately observable. Specifically, the detected cable is drawn as a green line overlay on the image output. This overlay provides a direct visual confirmation that the algorithm is tracking the physical cable rather than background texture or shadows.

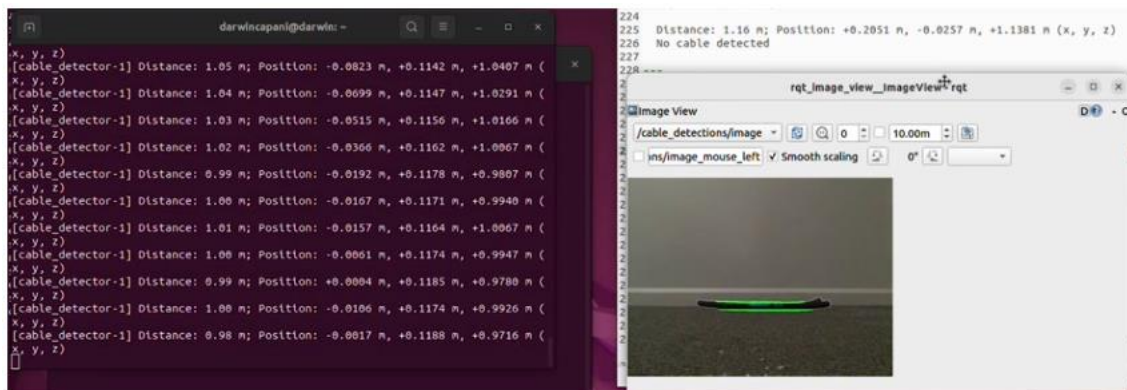


Figure 7: Example runtime outputs

Next to the overlay image, the node prints the computed distance and coordinates to the terminal each frame. This approach was an upgrade because a developer can validate that the overlay follows the cable and the numerical coordinates change smoothly and consistently as the cable moves in front of the camera.

Ease of System Integration

The package was upgraded to publish outputs using ROS 2 topics, enabling other nodes to consume cable detection results without modification. The output interface includes an image topic that carries the overlay visualization and a data topic that carries 3D location information.

The intent is to provide a clean separation between perception (detecting and localizing the cable) and decision-making (how the robot uses that location for path planning or interaction). Additional integration-focused upgrades include consistent message semantics (units in meters, camera-frame coordinate conventions documented in-code), explicit handling of the ‘no cable detected’ condition, and informative logging to help integrators quickly diagnose input-topic misconfigurations or sensor issues.

Depth Alignment and Missing/Noisy Depth at Cable Pixels

Accurate 3D localization depends on correct alignment between color pixels and depth samples. Without aligned depth, a pixel selected from the color image can map to a different physical location in the depth frame, producing incorrect distance and coordinate estimates. Even with alignment enabled, thin objects can present missing depth values due to sensor resolution limits, specular surfaces, or the cable occupying only a small fraction of a depth pixel footprint.

The package uses aligned depth registered to the color image, and the localization step is designed to tolerate imperfect depth measurements. Thus, this requires rejecting invalid depth values, smoothing or aggregating samples along the detected cable pixels, and ensuring that the reported coordinates remain stable and consistent as the cable moves. These considerations directly informed both the implementation and the evaluation procedure.

Converting Pixels to XYZ Coordinates

Transforming a detected cable pixel location into a metric 3D point is not a direct readout of the depth image; it requires projective geometry. The system must use the camera’s intrinsic

parameters (focal lengths and principal point) to convert from image coordinates (u , v) and depth (Z) into camera-frame coordinates (X , Y , Z).

This step is sensitive to calibration consistency: the intrinsics used must match the active stream configuration, and the depth units must be interpreted correctly (typically meters after scaling). To keep this computation reliable, the package maintains a single source of truth for intrinsics and depth scale via the camera driver, and it documents the coordinate frame used for output. The terminal output shown during runtime (Figure 16) was used to double-check the sign and magnitude of the computed coordinates as the cable was moved in controlled directions.

ROS 2 Timing, Synchronization, and Throughput

Cable detection has to run continuously and respond to motion, so timing and processing speed matter. A common failure in RGB-D pipelines is using color and depth frames that aren't synchronized—this makes the depth values come from a different moment than the cable pixels you detected. Another common issue is backlog. If the node can't process frames as fast as the camera publishes them, messages pile up, latency grows, and the overlay/XYZ output starts lagging behind the real scene.

This package reduces those risks by using a consistent, predictable callback flow and by sampling from aligned depth that corresponds to the same color frame used for detection. The terminal logs and the green overlay also make timing problems obvious (for example, the green line will visibly trail behind the cable if latency builds). Finally, launching the RealSense and cable-detection nodes together helps prevent configuration mistakes, like mismatched resolutions or subscribing to unaligned topics, that can break alignment and accuracy.

“No Detection” Handling

While the package is running, there will be frames where the cable is not in view or detection is temporarily unreliable (e.g., due to motion blur). The software must handle these cases explicitly rather than emitting stale coordinates. The package includes a clear ‘no cable detected’ condition, reflected both in terminal logging and when no coordinate outputs are published. This makes downstream integration safer because other autonomy components can gate their behavior on detection validity.

Below, the concept selection matrix for cable detection subsystem is located and shows which of the concepts, defined previously, is most suitable for meeting the requirements of Fall 2025 Aircraft Robotic Assembly.

Table 7: Cable Detection Concept Selection Matrix

#	Requirements	Metric / Unit of Measure	RealSense RGB-D (Aligned Depth)	RGB Camera Only (2D Vision)	Stereo Camera (Depth from Stereo)	2D LiDAR + Floor Model
1	Must provide real-time cable detection updates.	≤ 100 ms per frame	Can achieve the targeted unit of measure in under 100 ms with an efficient pipeline.	Can achieve the targeted unit of measure in under 100 ms with simple processing.	Can achieve the targeted unit of measure, but depth computation can add latency.	Can achieve the targeted unit of measure, dependent on scan rate and filtering.
2	Must output cable location in usable 3D coordinates for autonomy.	Publishes (X, Y, Z) in meters	Meets the targeted unit of measure by sampling aligned depth at cable pixels and converting to XYZ.	Does not reach the targeted unit of measure because it cannot produce true XYZ without depth.	Meets the targeted unit of measure by estimating depth from disparity and converting to XYZ.	Does not fully meet the targeted unit of measure because it primarily returns 2D range data.
3	Must enable quick operator	Overlay image	Meets the targeted unit	Meets the targeted unit	Meets the targeted unit	Does not reach the

	validation of detection correctness.	with cable marked	of measure by drawing the detected cable as a green line overlay.	of measure by drawing the detected cable as a green line overlay.	of measure by drawing the detected cable as a green line overlay.	targeted unit of measure because it has no image overlay without a camera.
4	Must remain reliable when depth is missing/noisy on thin objects.	Reject invalid depth and keep stable output	Meets the targeted unit of measure by rejecting invalid samples and aggregating depth along cable pixels.	Does not reach the targeted unit of measure because there is no depth validation.	Partially meets the targeted unit of measure, but thin/low-texture cables can reduce disparity reliability.	Does not consistently meet the targeted unit of measure because thin cables may not return stable LiDAR hits.

4.5 Gripper Re-design

The concepts selected for the cable detection were chosen based on user needs, requirements, implementation cost, and speed. The chosen concepts address the main issues identified by the previous semester, while pertaining to the client's needs and requirements.

Increase Servo Current

This option is the least intrusive and keeps all the existing physical elements of the robotic arm and by retaining the existing design, this reduces integration time and complexity. Without knowing the exact gear ratio reduction of the servo motor, only an estimation of the torque produced by the motor inside the servo can be made. By using the formula $\tau_{\text{motor}} = K_t \cdot I$, and given stall torque and current values from DYNAMIXEL (10.6Nm and 4.4A) [17] we can determine the torque constant to be 2.4A/Nm. Then using this torque constant, we can find that just by increasing the current from 4.4A to 5A, the torque is increased to 12Nm.

Servo Upgrade

This option would involve a teardown of the arm and gripper assembly in order to replace the servo motor with one with a higher stall torque. A higher stall torque means that there is more force being applied to the gripping jaws, preventing the cables from slipping. However, the servo motor being currently used in the arm already has the highest stall torque available. In addition, these servo motors are very expensive, ranging from 400-600 dollars [17], making this the most expensive option.

Gripper Redesign

This option requires no modifications to the electronics of the arm as well as the system that controls the actuation of the arm and gripper. By taking inspiration from a Fractal Vise, a vise that uses independently pivoting crescents to adjust to the shape of objects placed between the two jaws. By conforming to the shape of the object between the jaws, in this case a cable, there is a greater surface area in contact with the cables, reducing the chance for it to slip and fall out of the jaws. The design will be 3D printed, allowing for fast development and iteration to fine tune the design.

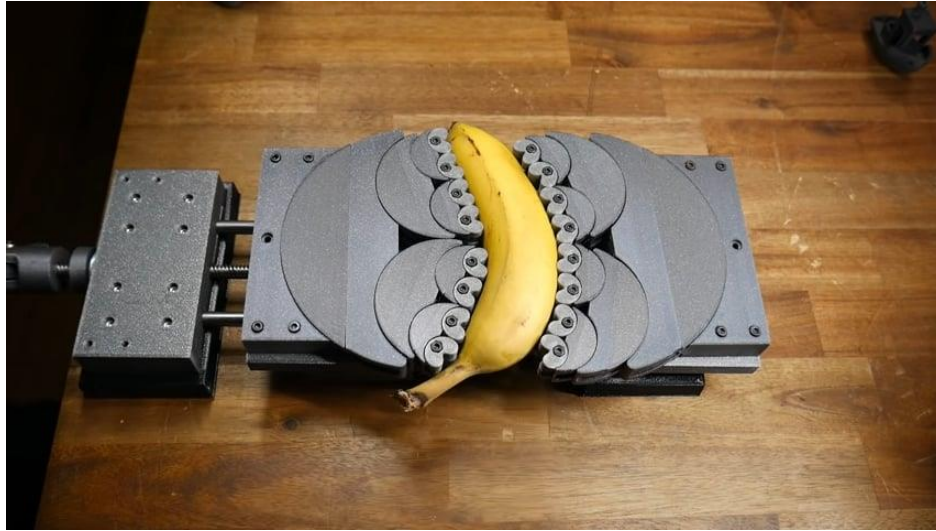


Figure 8: A 3D printed model of a fractal vise [8]

Table 8: Gripper Re-design Concept Selection Matrix

#	Requirement	Metric / Unit of Measure	Increase Servo Current	Fractal Vise gripper	New Servo + New Gripper
1	Ease of implementation	What needs to be changed on the arm	Change Code (Needs to be tested)	New 3D Printed Vise Gripper	Servo, Gripper, potentially entire gripper assembly
2	Minimum Time Cost	Time	1 Class time (change code + troubleshooting)	1 week of CAD + troubleshooting (2 weeks)	3-4 weeks (delivery, printing, testing, troubleshooting,
3	Increased Reliability	Cable Pickups	More than initial testing	More than initial testing	More than initial testing

4.6 Pathfinding Algorithm

The concepts that are selected for improving pathfinding were chosen because they meet the team's needs and requirements for testing. Each concept is aimed at addressing problems with and improving upon the existing autonomous code written and implemented by Spring 25, Fall 24, and beyond. The actual implementation of Fall 25's pathfinding algorithm was chosen from these concepts based on their feasibility as well as their ease of integration with the previous semester's work.

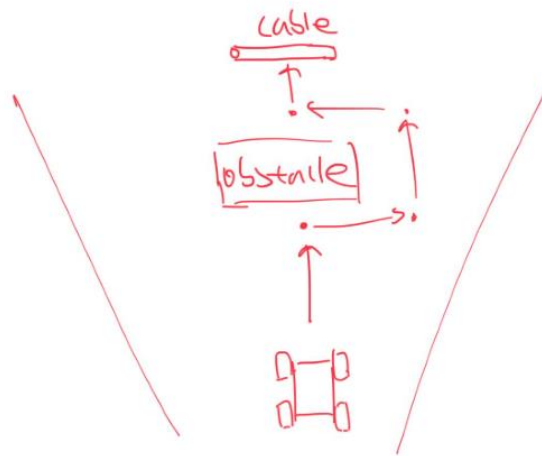


Figure 9: Reactive Pathfinding

Reactive pathfinding program is basically where the robot decides on where to move based on information sent to its computational unit by the LiDAR, whether to avoid an obstacle, move towards the cable, or just moving straight. This path-finding algorithm takes advantage of the pre-built computer vision algorithm used for cable detection as well as the LiDAR to have a

constant update of the robot's surroundings allowing the robot to make decisions. In Figure 12: reactive pathfinding, when autonomous mode is first initialized, the robot will consider what the objects are close to it and can be seen by the LiDAR and will begin moving forward. The robot will always be programmed to move forward if there is more than one option, because having the robot explore multiple directions consumes too much time and resources. If the robot senses an object that is on its trajectory forward, it stops and calculates out the next steps of action, basically where the robot would go next to avoid the obstacle. The robot would turn and avoid the obstacle from either the left or right side and then continue going straight. The calculations for turning should not involve any complex software, only some math tricks should be applied to achieve the desired result. From the diagram, the robot will continue going straight until the camera sees the cable causing the robot to go in that direction and reach it. The robot will remain in place if there is no path forward and all directions is blocked by obstacles. This method for

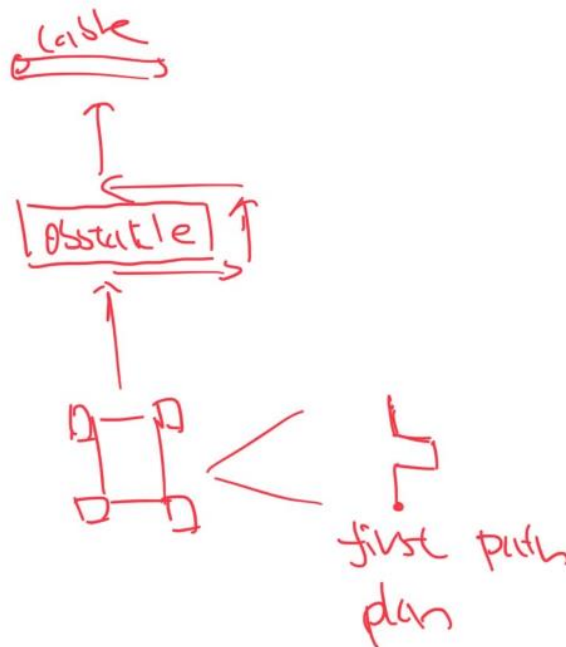


Figure 10: Deliberative Pathfinding

obstacle avoidance is more crude and less accurate, but can be accomplished with very minimal

resources, making it feasible to implement, easy to debug, and trivial to create new features on top of it.

Deliberative pathfinding is an approach where the robot takes in a scan or a map of the environment beforehand and just path plans to reach the end goal. This approach will have robot computing and finding the best path possible beforehand and begin executing after the most optimal path is determined. The robot will begin by receiving an input of a set of data or a plain image, or any representation of the environment in which the robot will operate that contains the location of the robot, the obstacles, and the cable. The input will be parsed into a type of representation that the computational unit can understand, then path-planning algorithms such as Dijkstra's algorithm, flood-fill algorithm, etc. can be used to figure out the most optimal path to the cable. The speed and direction of the robot can all be pre planned making autonomous mode extremely efficient in both speed and consumption. This approach can create a highly accurate autonomous mode allowing the robot to seamlessly avoid all static obstacles and reach the cable in the most optimized time, making it more preferable to other path-finding approaches. This approach, however, assumes that the environment can be mapped out initially by an external source outside of the robot.

The return home feature is an extension built on top of the existing autonomous mode operations that allows a robot to return the robot to the position where autonomous mode was

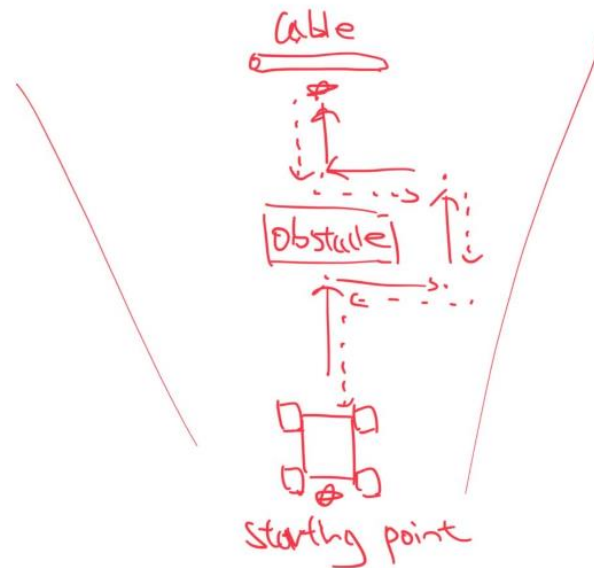


Figure 11: Return home feature

first initialized. If the autonomous mode uses reactive pathfinding, then the return home feature will need a compass to store the direction in which the robot is going, the rpms of each wheel, as well as the duration that the robot travels in without switching directions. When the robot is needed to return home, after it finishes its operations on the cable, in this semester, return to home will be activated after the robot picks up the cable, the robot will reverse the stored direction and use the stored time and speed to trace its way back to the starting point. If the pathfinding algorithm is deliberative, then the robot can just use the most optimal path generated by the algorithm and follow it back to the starting point using the same method as the reactive algorithm by storing the relevant information. The return home feature fits well with the needs and requirements as improves the safety aspect of the robot, having the robot return home is helpful as an end to autonomous mode because the current configuration can only get to the cable and cannot do anything after that.

Below, the concept selection matrix for the pathfinding algorithm subsystem is located and showed which of the concepts, defined previously, is most suitable for meeting the requirements of Fall 2025 Aircraft Robotic Assembly.

Table 9: Pathfinding Algorithm Concept Selection Matrix

#	Requirements	Metric/ Unit of Measure	Reactive Pathfinding	Deliberative Pathfinding	Return Home Feature
1.3	Help improve localization accuracy using LiDAR and IMU sensor fusion	$\leq 5\text{-}10$ cm	Can achieve the targeted unit of measure.	Very accurate as image of the robot's surroundings are given.	Can be achieved through sensor-based detection similar to reactive pathfinding.
4.2	The robot should avoid obstacles through sensor-based detection.	$\geq 95\%$ accuracy	Can be achieved for static obstacles and some dynamic obstacles depending on the speed of the dynamic obstacle.	Can perfectly avoid static obstacles, however approach is not sensor based.	Can be achieved through sensor-based detection similar to reactive pathfinding.

Table 9 introduces the concept selection matrix and shows which concept best meets the standards set for Fall 2025 aircraft robotic assembly. The reactive pathfinding can meet both requirements; the deliberative pathfinding will be easier to implement for the first requirement; however, the second concept will not meet the second requirement, as it explicitly asks for sensor-based detection. The third concept of the return-to-home feature depends heavily on how autonomous mode operation is implemented, either reactive or deliberative. Therefore, it is

determined that the concept chosen to implement for this semester is reactive pathfinding using LiDAR data to detect and prevent collisions and improve the robot's localization.

4.7 Safety Features

The Safety Features subsystem concepts were selected to support the safe operation of the robot during navigation and testing, while being in-line with industrial standard. The subsystem consists of two core components, the Emergency Stop button (E-Stop), and ultrasonic range sensors. Each component addresses safety from a different perspective, with the E-Stop being a physical cutoff switch and the ultrasonic sensors helping with environmental awareness. These two components were chosen based off their reliability, response time, and ease of integration to the overall structure of the robot.

Table 10: Safety Features Concept Selection Matrix

#	Requirements	Metric/ Unit of Measure	HC-SR04 Ultrasonic Sensor	TORMACH Estop Twist Lock
1.2	Learn and map the working environment using sensor data	$\geq 95\%$ (mapping coverage accuracy)	95–98% accuracy	NA
4.1	Robot should have a power cutoff switch	≤ 1 second (turn-off time)	NA	5–30 ms
4.2	Robot should avoid obstacles through sensor-based detection.	$\geq 95\%$ accuracy	95–98% accuracy	NA
8.4	Avoid obstacles	≥ 0.5 ft	0.8 – 157 inches	NA

The Emergency Stop button consists of the leads from the battery, a NC (normally closed) switch, and the motor controllers for the wheels on the base of the robot. When the button is pressed, it immediately disconnects the power from the motor controllers. Since it does not interrupt the power to the Raspberry Pi, it must be informed of its activation in order to run emergency signals on the LEDs, tell the operator there is an error, as well as any other emergency protocol that may be added to it in the future.

A proposed design solution uses two pins on an Arduino Nano connected to the other set of contacts on the Emergency Stop button. This makes it so when the power is disconnected, the power between the two Arduino pins is also disconnected, letting the Arduino know to send a distress signal to the Pi.

Ultrasonic Sensor

The ultrasonic sensor provides continuous distance measurements to detect nearby obstacles that might be in the blind spot of the LiDAR system. This system integrates easily with the Arduino used for the Emergency Stop button, but it may require software smoothing due to potential interferences or angled reflections. Since the sensors operate at 5V and the Arduino has a 5V pin, it should work completely off the power of that system.

Technical Risks

Key design risks include electrical isolation between subsystems, timing reliability for safety messages, inaccurate readings caused by environmental interference, and delays in ROS processing for emergency signals. The system must detect unsafe conditions within milliseconds, requiring careful tuning of message frequency, sensor filtering, monitoring on the Raspberry Pi.

The Safety Features subsystem will undergo unit testing for each component independently before integration with the rest of the robot.

Emergency Stop Button Testing

The Emergency Stop button will be mounted on the back of the robot, and it will be wired between the battery and motor controller for the wheels. The status LED on the motor controller will act as an indicator for power in the system, and when the button is pressed, it should immediately turn off. The Arduino Nano will be used with the E-Stop and will have an LED turn on when the button is pressed to show the distress signal being sent to the Raspberry Pi.

Ultrasonic Sensor Testing

The ultrasonic sensors will be put on a moveable cart and evaluated at two positions: A clear path in front of the sensor, and an obstructed path within 6 inches of the sensor. The Arduino will measure distance values and send an alert to the Raspberry Pi when a close object is detected. Proper function will be confirmed when the robot stops the wheels and a ‘call for help’ signal is triggered.

5 System Proposed Design

5.1 System Upgrade Final Design

The final design of the system upgrade consisted of replacing the Pi 4 with a Pi 5, and using emulation to get around software incompatibility issues. When the project was first handed to the team, the robot was running ROS2 Humble on Ubuntu 22.04, the latest version supported by the Pi 4. The Pi 5 only supports Ubuntu 24.04, which only supports ROS2 Kilted or Jazzy [2]. Because different ROS2 releases aren't forward/backward compatible, it is critical that your system will support the version of ROS2 that the system was built on. Previous groups have tackled this issue by using Docker [3], but were not able to install all the necessary dependencies to be able to use new hardware for the project. Thus, the previous teams stuck with the Pi 4 until the Spring 25 team hit a strong bottleneck in performance as reported in sections 4.4 and 5.5 of their system design report [4]. With the much more recent Pi 5, which supports an updated version of Docker, the Fall 2025 team has not experienced enough difficulty to justify not moving forward with this option. There are some issues with keyboard passthrough, meaning keyboard control of the robot is no longer a feature. Joystick control remained, meaning the feature of tele-operation still remained. Dependencies for the arm are still to be compiled.

The key design parameters for the system upgrade was compatibility. Given the short amount of time and limited programming manpower that the Fall 2025 team has, it is imperative that old software was able to be tested and recycled. By choosing a Pi, the hardware layout remains relatively unchanged; and by using Docker, the software development environment is kept very similar. And with docker's minimal performance hit [5], the Fall 2025 team can

harness the increased computing power of the Pi 5 to build more effective camera detection and pathfinding algorithms.

5.2 Visual Feedback Final Design

The final design for installing the Visual Feedback system involves installing aluminum angles on the side of the robot's arm base to allow the component to sit flush and be seen by users during testing. The top robot base, which is the same base on which the arm is located, measures 10.25 inches in length and 9 inches in width. The aluminum angle is a 3-foot piece that was cut into specific lengths to install on the base.

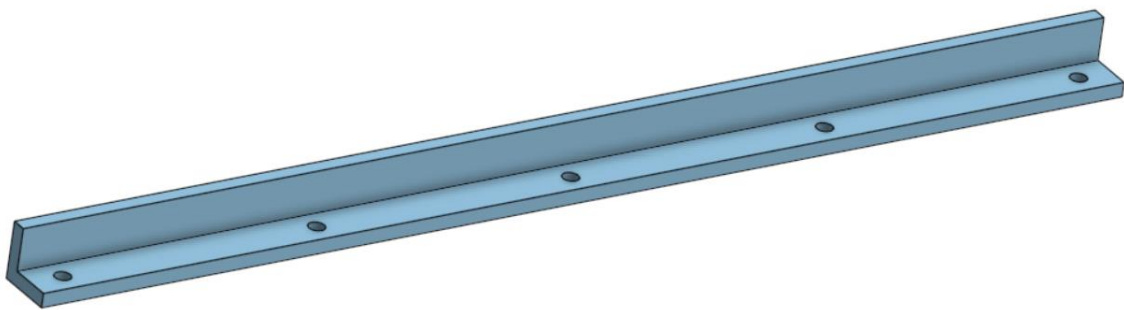


Figure 12: Aluminum Angle CAD Drawing

The aluminum angles outside measure 0.5 inches, which is equivalent on both sides. This piece was cut using the tools in the Design Lab to fit the dimensions of the arm base. The angle should be able to lie flush against the flat part of the arm base, while the LED is attached to the side that is facing out, not facing upwards. To install the angles, the existing holes on the arm base needed to be used for nut and bolt connections. There are 9/64-inch drilled mounting holes selected to provide a secure attachment. Each side of the arm base contains 5 different holes, and

these were used as a reference to cut holes in the aluminum angles. The final result with all connections in leaves a secure mount with minimal weight added to the total mass of the robot.



Figure 13: Angle Installation on Arm Base

The electrical connection between the WS2812 LED Strips and the Raspberry Pi is quite simple. A total of 140 LED IC chips were used on the component for maximum visibility on the robot. This LED variation requires a 5-volt power source, which the Raspberry Pi supplies through a few pins on the board. The LED is connected through pin 12, also known as GPIO 18, as well as ground. GPIO 18 acts as the controller for the LEDs and is one of the four PWM pins on the Pi board. PWM, which stands for Pulse Width Modulation, is commonly used in applications for controlling colors and patterns within RGB LEDs. These LEDs require their ground connection to be on the same terminal as the Raspberry Pi ground, because the application cannot be controlled without this step. The programs for running test outputs on the component are done through the Pi software, where it reads the requested logic for which

colors/outputs should be used, and it will output accordingly. Software control is implemented in Python, where the LED strip is logically divided into segments using indexed arrays. Each segment corresponds to a physical region on the robot base and a specific functional category, such as system mode indication, safety alerts, or command acknowledgment.

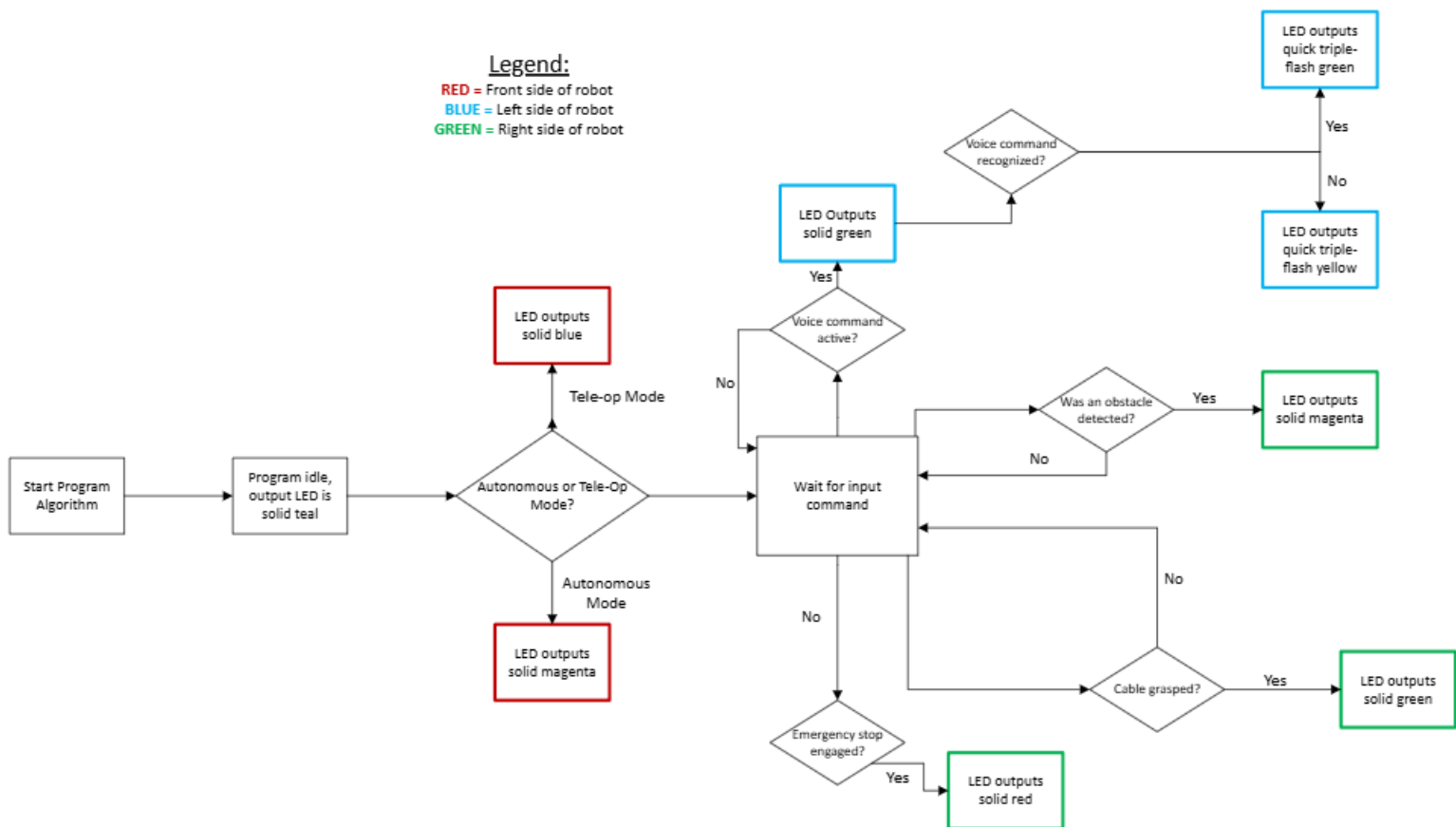


Figure 14: Flow Chart for Final Visual Feedback Design

From a manufacturability perspective, the design relies on commercially available LED strips, standard aluminum stock, and basic fasteners, minimizing cost and fabrication complexity. The design is also serviceable, as individual LED segments can be replaced without removing the entire assembly. The final implementation meets the project’s visual feedback

requirements by providing immediate, unambiguous system state information that is observable during both development and live demonstrations.



Figure 15: LED Installation on Robot

5.3 Ramp Navigation Final Design

The final design of the navigation subsystem consists of three main components: The compass, the ESP32 board, and the compass mount. The compass mount, designed to hang below the robot to minimize magnetic interference, is mounted to the underside of the robot.

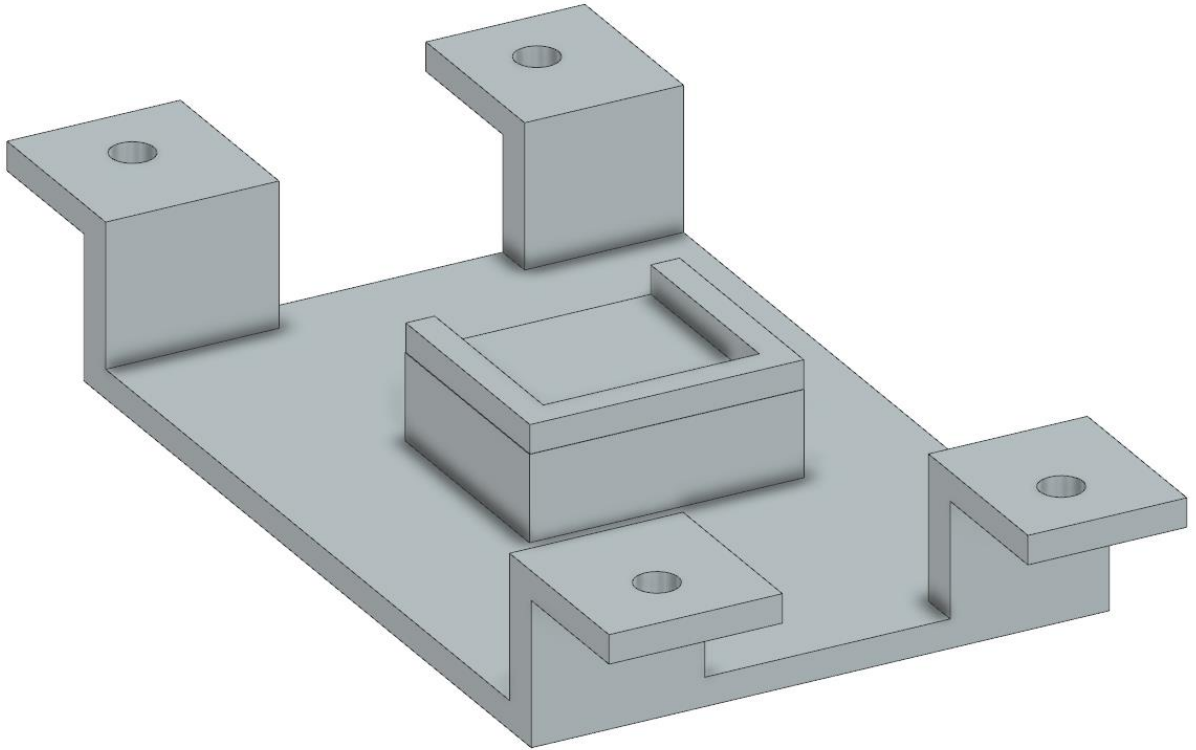


Figure 16: Compass mount CAD model

At only .7” in height, it does not pose any significant change to the ground clearance of the robot and is able to hold the compass away from the robot’s other electronics, minimizing interference. The compass mount is affixed to the bottom plate of the robot via the vents with nuts, bolts, and washers to prevent damage to the chassis.

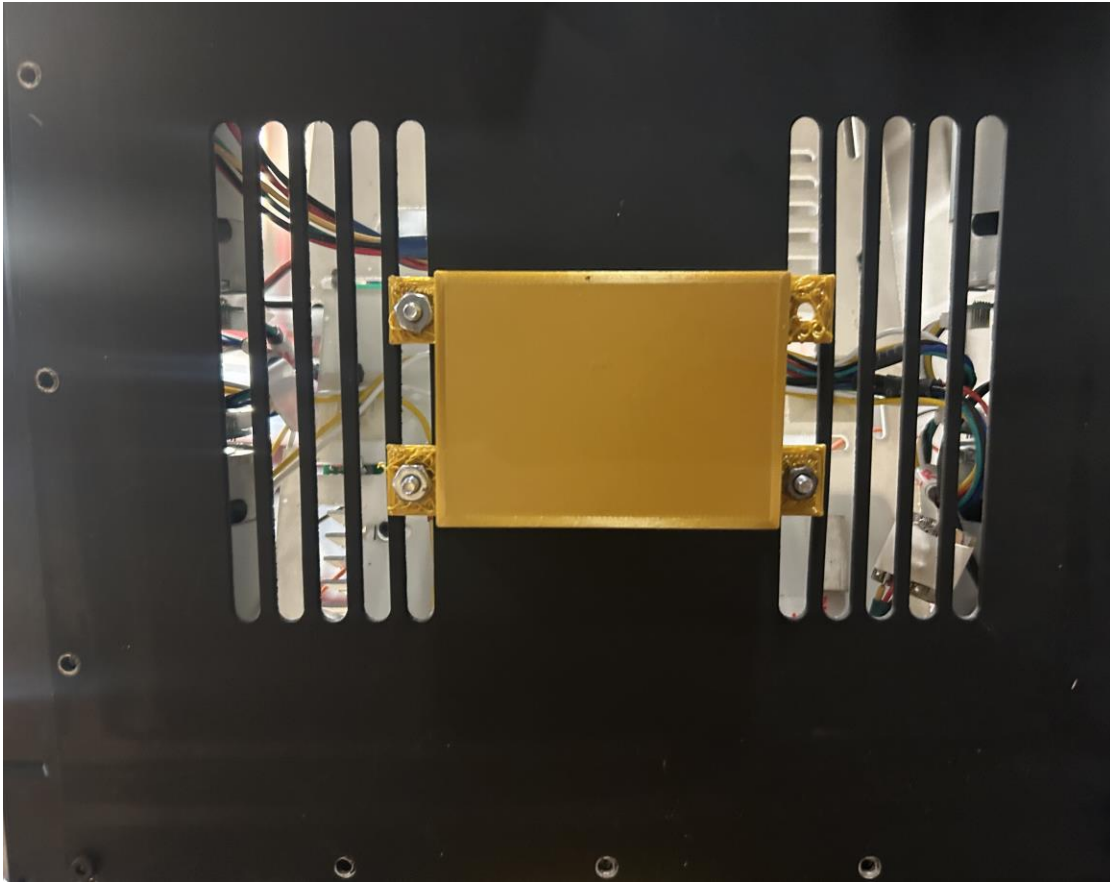


Figure 17: Compass mount installs

The compass slots into the center platform where it is affixed with an adhesive, keeping the compass as close to the center of the vehicle as possible. The wires from the compass are threaded through the vents into the body of the robot to the ESP32 board. The ESP32 board is wired to the PI 5 and the system is complete. (A stipulation to the robot, right now the compass chip being used does not belong to the group, so the compass chip has not been adhered to the compass mount.)

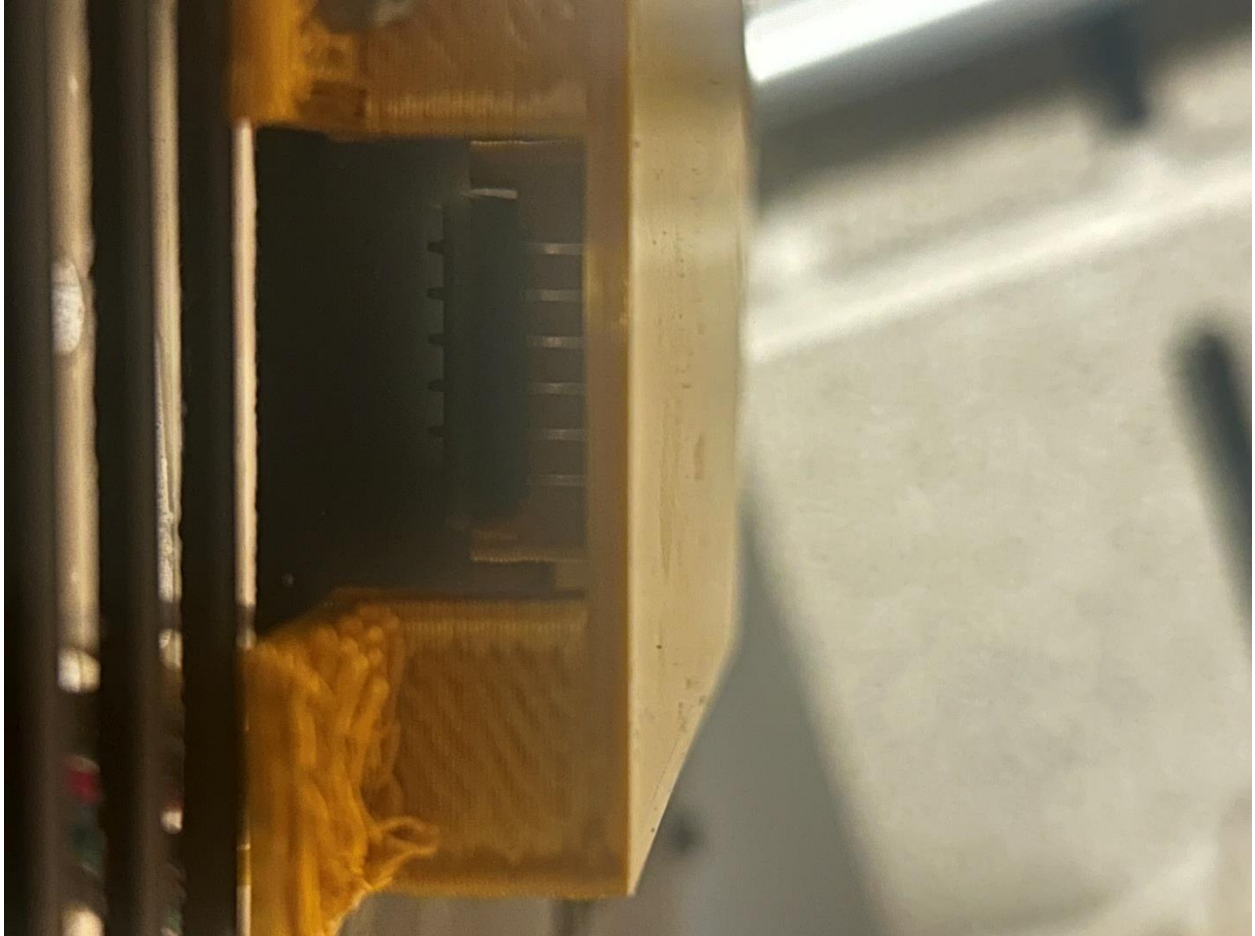


Figure 18: Compass chip in mount

For the code, the compass communicates the bearing to the ESP32, which sends the data to the PI 5 where it is constantly updating in the motor driver code. The motor driver code controls all wheel motions performed by the robot, now including corrective turns to prevent veering off course (code not implemented.) The bearing from the compass is constantly read, and when the robot is found to be outside of the acceptable error, PID motor control code returns the robot to its correct bearing. To be effective, the code must be fast and accurate to react in time to stimuli and not over/under shoot on its corrections.

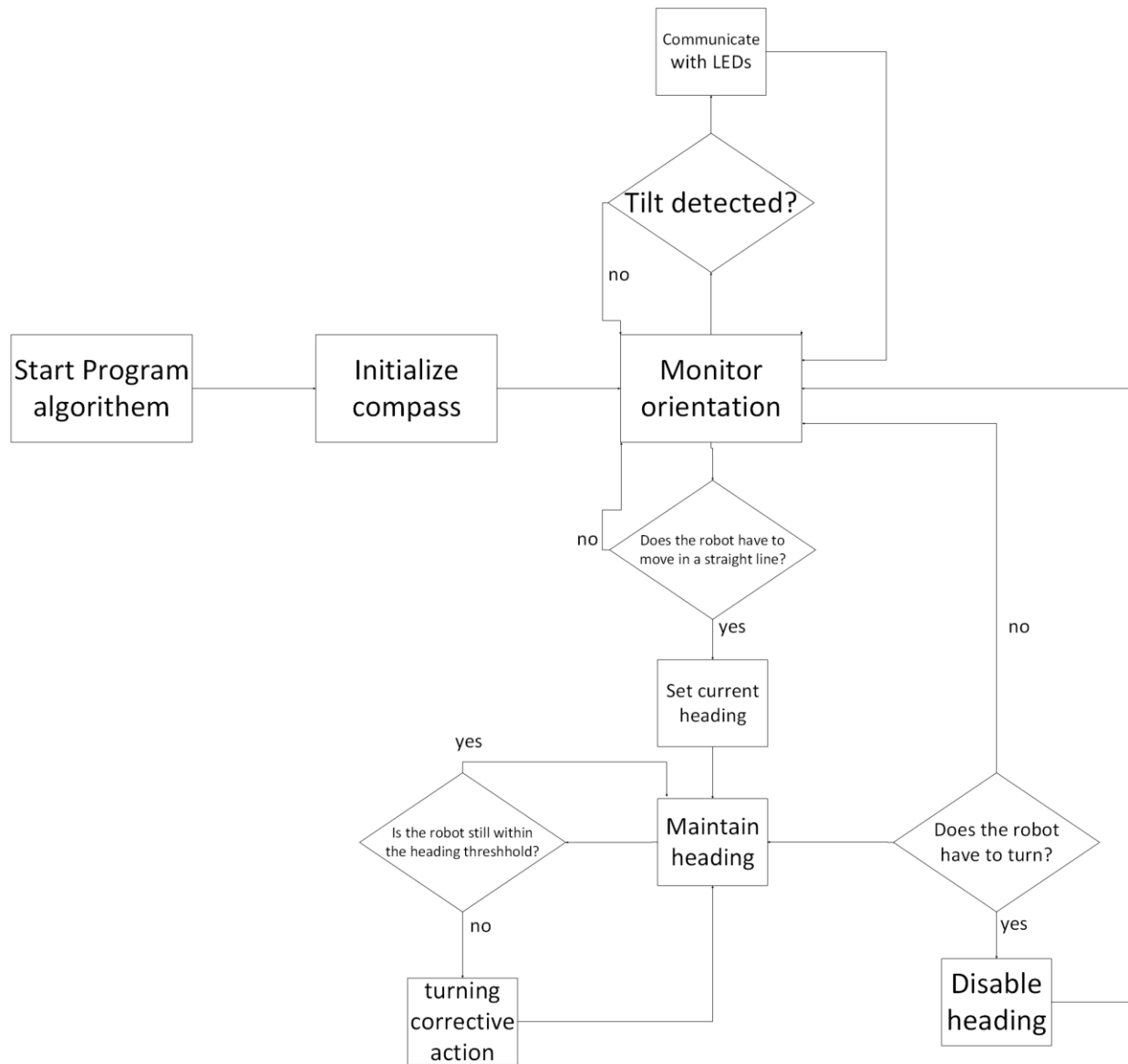


Figure 19: Flowchart for code logic

5.4 Cable Detection Final Design

The final design for the Cable Detection ROS 2 package consists of a real-time perception node that detects a cable in the camera's view and localizes it in 3D using an Intel RealSense RGB-D camera. In the final hardware configuration, the RealSense camera is mounted on the front side of the robot's base, facing outward so the system can observe the

workspace directly in front of the platform during operation. The package is designed to produce both visual confirmation and quantitative localization output at runtime. A green line overlay is drawn directly on the RGB image to show the detected cable, while the system simultaneously computes distance and XYZ coordinates from the aligned depth stream. This final configuration ensures the detected pixels and sampled depth correspond to the same scene content, allowing the output to be interpreted reliably by a user and consumed by other autonomy modules through ROS 2 topics.

The software architecture of the final design is based on a two-node pipeline. The `realsense2_camera` node publishes the RGB stream and the aligned depth-to-color stream, and the custom cable detection node subscribes to these topics. Each frame follows a fixed processing flow: preprocessing is applied to stabilize the image, the cable is isolated using edge/contour logic, depth is sampled only at the detected cable pixels, and those pixel-depth pairs are converted into 3D points using the camera intrinsics. The resulting cable location is published as a visualization image with the green overlay and as numeric localization data (distance and XYZ coordinates) that is printed to the terminal and optionally published as point data for downstream use. This structure supports continuous operation while keeping the outputs synchronized and interpretable during motion.

The final design prioritizes consistent real-time behavior. Cable detection must respond immediately as the cable moves, which places constraints on throughput and message handling. If processing cannot keep up with the camera publish rate, latency builds and the overlay and coordinates lag behind the physical scene. If color and depth are not properly aligned, depth samples can map to the wrong pixels, producing incorrect XYZ values even when the overlay appears correct. The final implementation mitigates these issues by using the aligned depth-to-

color topic, maintaining a predictable callback flow, and using runtime visualization to reveal mismatch conditions quickly. The ability to launch the camera and detector together also reduces configuration mistakes such as incorrect topic selection, resolution mismatch, or frame misalignment.

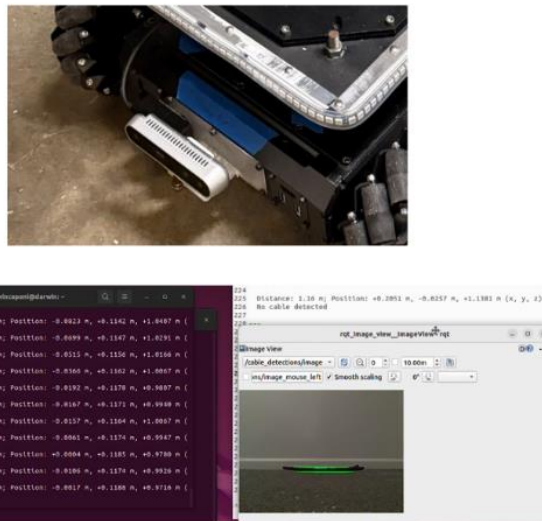


Figure 20: Cable detection camera position and terminal results

5.5 Gripper Re-design Final Design

The final design for the Gripper is a 3D printed fractal gripper design with 3 levels of articulation. The fractal portion of the gripper was integrated into the original design of the parallel plates for ease of integration.

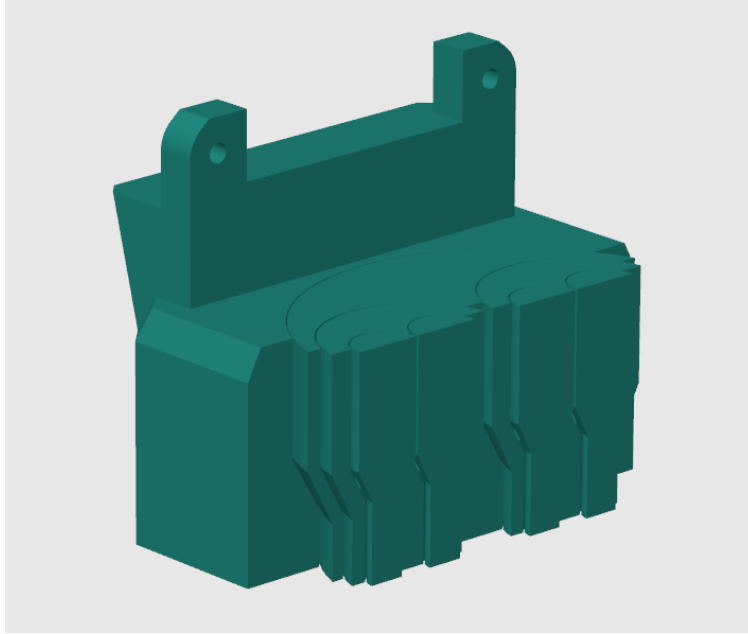


Figure 21: New Gripper CAD design

The gripper is printed in PLA and was chosen for its availability and ease of printing. Unlike other materials such as ABS or ASA, PLA does not release toxic VOC fumes and does not require a heated enclosure to prevent material warp. Due to molding the fractal portion into the original gripper plates, the design integrates seamlessly into the arm.

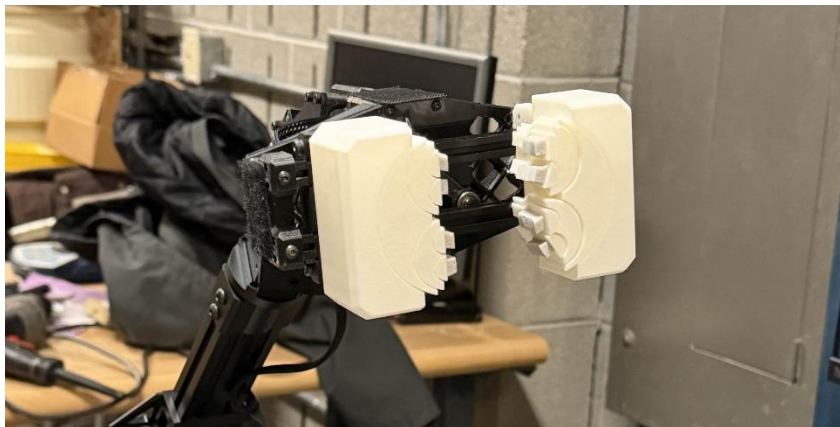


Figure 22: 3D printed gripper mounted on robotic arm

5.6 Pathfinding Algorithm Final Design

The final design for the path-finding algorithm subsystem consists of an implementation of A* algorithm. The implementation when the program receives data on obstacles, the grid resolution, the robot radius, the grid limits, as well as the starting and end points. The algorithm will then construct a grid and populate it with Boolean values to represent the obstacles by expanding the obstacle points depending on the radius of the robot, while also setting up the eight directional movements that a robot can make. Internally, the robot's position and cost are stored inside of a nested class object and a function is created to convert the robot's position on the grid to the robot's position physically. The actual path planning implements the A* search algorithm with two containers, one for steps already explored, and one for potential steps to be explored. The algorithm will continuously explore neighboring positions and will update if the cost, the distance to the end point, is cheaper. Once the end point has been reached, the algorithm will backtrack to collect all points on the most optimal path, convert them into real world coordinates, and then store and return them all as a list.

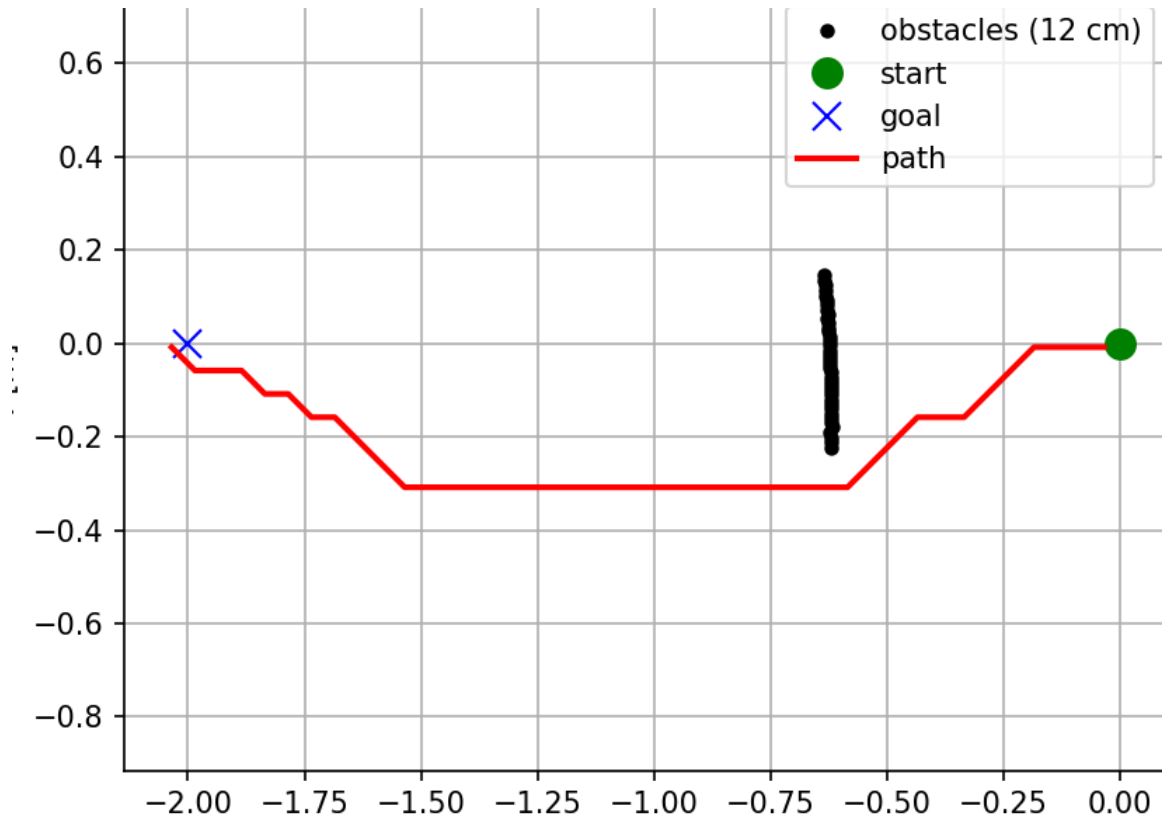


Figure 23: A* algorithm implementation

The full autonomous mode code utilizes the A* search algorithm implementation above, to generate a path and then have the robot follow said path will updating the path if any obstacles were to interrupt said path.

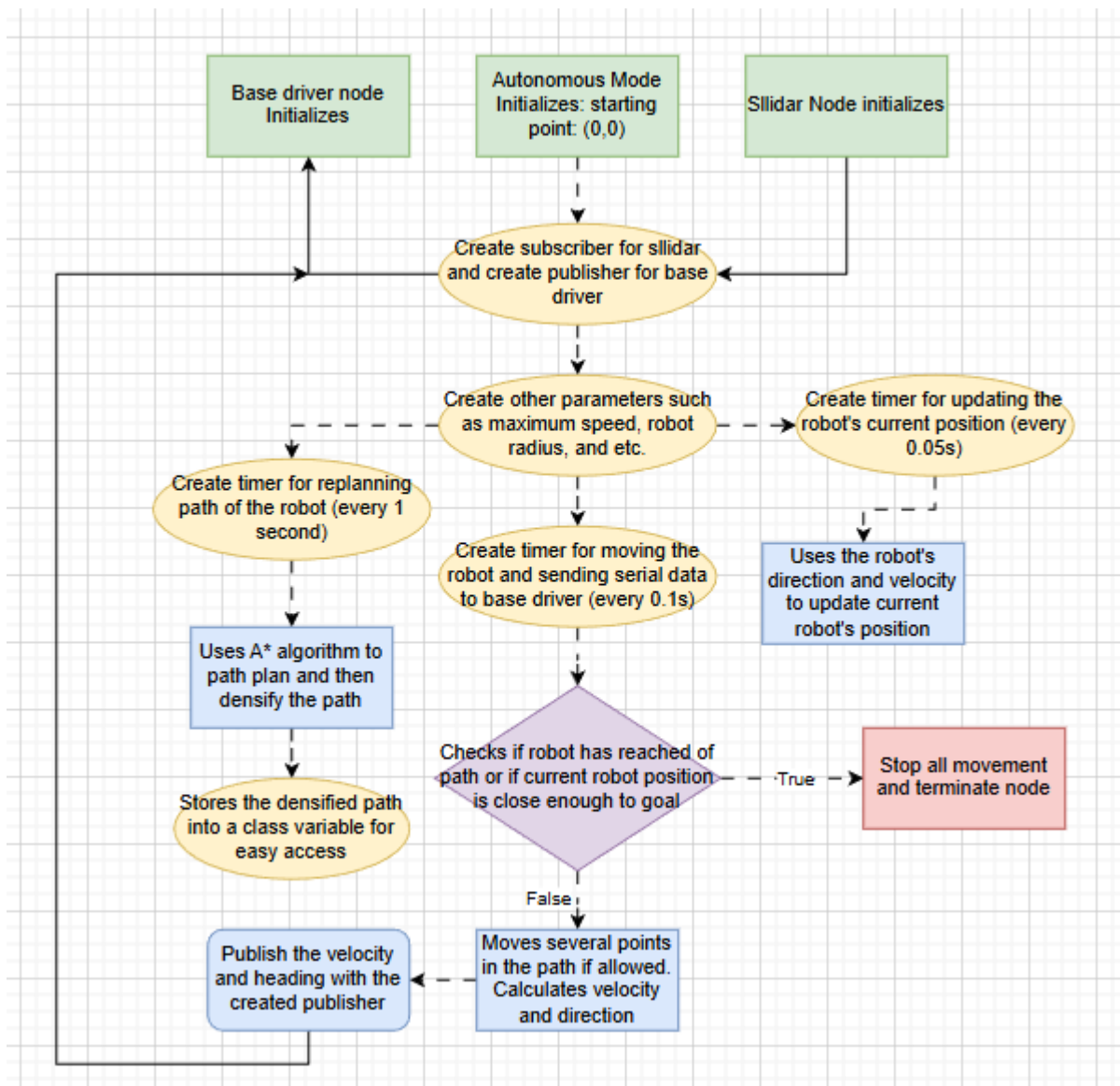


Figure 24: Autonomous Mode

The code requires three packages to run, the base driver package which controls the movements of the robot, the sllidar package which runs and publishes LiDAR data, and finally, the base_teleop package with the autonomous and A* algorithms. The autonomous mode algorithm will create a publisher for the robot's velocity and heading and a subscriber to listen to the sllidar outputs. Then it will create other parameters used in the code such as the velocity of

the robot. Lastly, the program will create three timers: path replanning, robot movements, and positional updates. The path replanning timer will replan the path every second to update the robot's future movements, since the robot will constantly be moving thus the obstacles positions relative to the robot and the robot's own position will have also been updated. If viable path was found, then the robot will set its velocity to 0 and remain in place. The robot movements timer will check if the robot has reached its goal, if the goal has been reached, then the robot should stop moving and shutdown, if the goal has not been reached, then the robot will move forward eight points along its pre-planned path and update the robot's velocity and heading through the already created publisher, this timer activates every 0.1s. Finally, the positional updates timer will use the velocity and heading to calculate the current position of the robot every 0.05s, so that the path planning is accurate and not faulty.

5.7 Safety Features Final Design

The final installation of the safety features subsystem integrates both the Emergency Stop (E-Stop) button and the ultrasonic range sensors into the moving base and control architecture. The E-Stop button is mounted on the rear of the robot on the battery housing, allowing for quick activation to disable the wheels of the robot. The button is wired as a normally closed (NC) switch in series with the motor power lines, ensuring that when the button is pressed, it physically disconnects the power. This provides a hardware-level safety cutoff that does not rely on software execution which can have communication latency or lag. This lets the robot have a rapid and reliable response in emergency conditions.



Figure 25: Battery housing with emergency stop button and rear ultrasonic sensor

In addition to the physical cutoff switch, there are ultrasonic sensors mounted on the base of the robot, as previously stated. The purpose of these sensors is to detect objects within 6 inches of the robot's base; this is because this is the blind spot of the LiDAR system. Each sensor is fastened to the chassis with a strong Velcro-style tape; this allows the sensors to be mounted and easily swapped out if damaged, or a new orientation is wanted. These sensors are connected to an Arduino which processes the echo timing to determine the distance, flagging when the sensor is tripped. When an object is detected, a signal can be sent from the Arduino to the Raspberry Pi to alert it to the danger, and it can start appropriate safety measures.

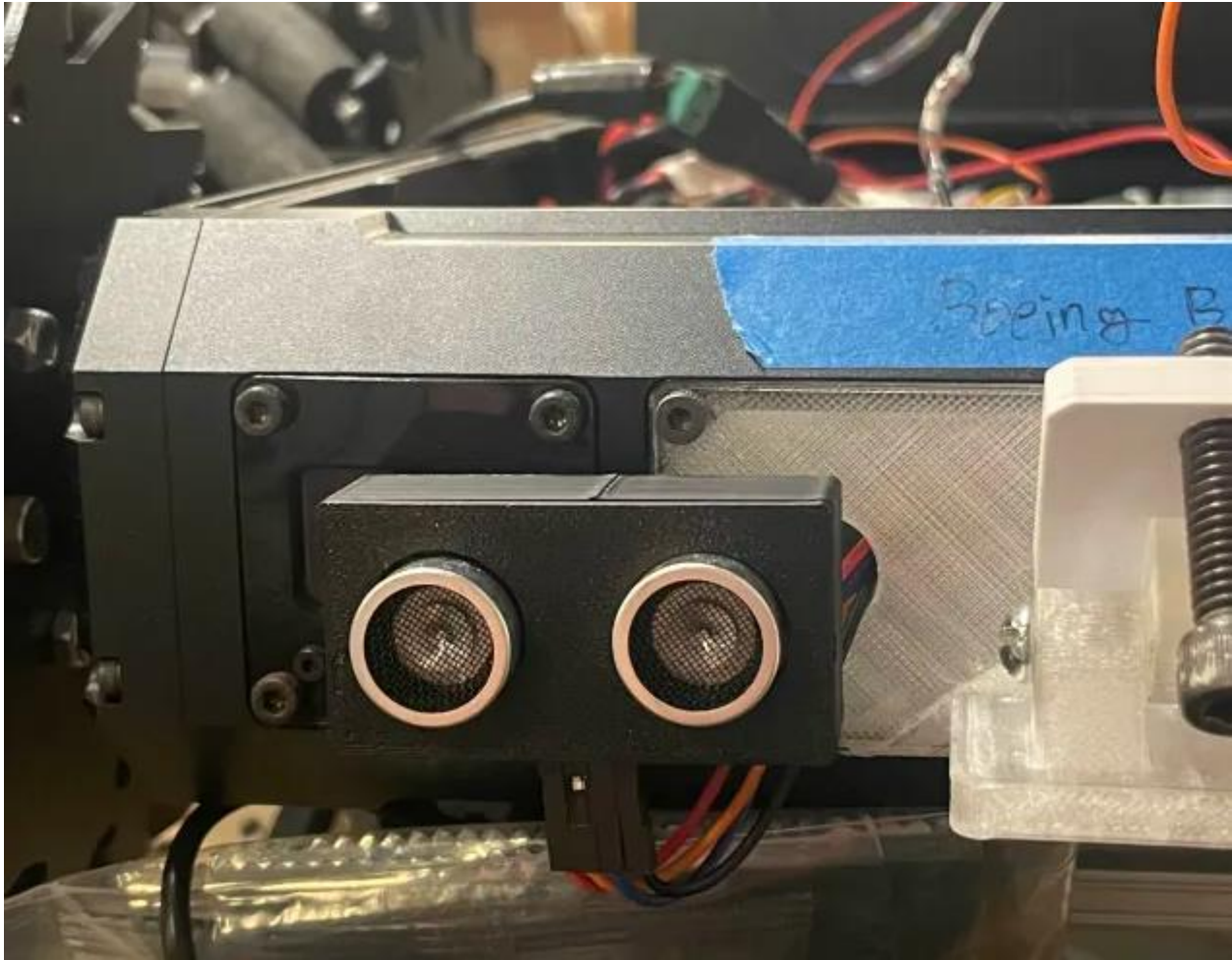


Figure 26: Ultrasonic sensor mounted to the front left of the robot

Together, the Emergency Stop button and the ultrasonic sensors form a double-layer safety system that combines physical intervention with automated environmental awareness. The final installation results in a lightweight safety implementation that enhances operator operation and obstacle detection while maintaining seamless integration with the robot's existing electrical and control systems.

6 System Evaluation

Table 11: Individual Tech Demo

Subsystem	Technical Lead	User Cases	Needs & Requirements	Test Description	Test Results
System Upgrade	Jay Tang	5. The robot should be able to avoid obstacles with precision	1.2 Learn and map the working environment using sensor data 7.1 Warn operator once battery decreases to a certain percentage 7.2 Inform operator when errors occur	The robotic base, now being controlled by the Pi 5, will be able to navigate around an oval track from joystick input by an operator with minimal need for corrections.	The robot was able to move around the oval track without any need for a human to physically realign the robot.
Visual Feedback	Brody Waters	8. The robot must respond to the user using visual feedback for actions completed	5.1 Must provide clear and immediate feedback to the user on robot's current state 7.1 Warn operator once battery decreases to a certain percentage 7.2 Inform operator when errors occur	The test will consist of inputting a 6-bit string to the terminal, where each bit represents a robot state. The bit number will be 0 or 1, and changing these bit numbers in a new terminal input will update the LED strip accordingly.	The LED output segments are sent to the correct rails. The voice command result bit is not active if the voice command status is not active. The emergency stop bit will overlap both the cable and obstacle detection if it's given an active state.
Improve Pathfinding	Peijing Xu	5. The robot should be able to avoid obstacles with precision	1.3 Help improve localization accuracy using LiDAR and IMU sensor fusion. 4.2 The robot should avoid static obstacles through sensor-based detection.	The robot should autonomously navigate an environment, a straight line, either without any static obstacles or with obstacles and the obstacles has to be seen by the LiDAR.	The robot was able to autonomously go from point A to point B when there are no obstacles in front of it. The autonomous mode has some issues when there are obstacles in front, it will pivot to avoid a stationary obstacle but will still end up hitting it.

Subsystem	Technical Lead	User Cases	Needs & Requirements	Test Description	Test Results
Ramp Navigation	Talon Carsto	3. The robot should be able to climb slightly to moderate inclines 4. The robot moves in the direction desired	1.1 The robot shall detect and navigate ramps in the environment.	The terminal will display the proper responses to stimuli performed on the test bot.	The compass was able to properly read its heading within a ½ degree and output the proper turning correction commands when deviations were simulated.
Emergency Features	Anderson Corriero	N/A	4.1 Robot should have a power cutoff switch 4.2 Robot should avoid obstacles through sensor-based detection.	The robot's emergency shut off switch functions properly	The Emergency Stop button works when pressed, stopping the wheeled base from moving. The ultrasonic sensors work at detecting obstacles within 6 inches, as well as sending out a signal to notify something was detected.
Cable Detection	Darwin Capani	1. Be able to recognize a cable in front of it 7. Show how far the cable is from the camera	6.1 Robot should be able to detect cable 6.2 Robot should output distance of cable	Camera will detect cable placed in front of it, and will output distance and coordinate location	The camera is able to see and detect the cables at a minimum distance of 0.3 meters. The measurements are very reliable, with about 1 and at most 2 cm of error. When detected, the cable portrays green pixelated lines in the camera output, and the terminal reads the distance and coordinates of this detected cable.
Gripper Design	Jeremia Silaban	2. Be able to grab various sizes of cable	10 Robot must be able to lift heavy objects off the ground, and securely grip aircraft cables with specific diameter	The gripper arm can lift heavier cables and securely grip cables of increasing	The robot was successfully able to grip cables up to 2.5" in diameter and hold them 10 in off the ground. In addition, the cable remained

Subsystem	Technical Lead	User Cases	Needs & Requirements	Test Description	Test Results
				diameter 0.5 < OD < 2.5 in	secure within the gripper with external disturbance (pulling, shaking)

6.1 System Upgrade

In order to test this subsystem, the robot's base (now controlled by the Raspberry Pi 5), will move be piloted around a oval track using joystick inputs. The base will be able to travel forward in a straight line without the need for drift corrections, and will be able to navigate curves. This will demonstrate that the firmware for controlling was successfully migrated onto the new robot, and that the new upgraded processor will be able to move the robot after the pathfinding algorithm generates a path for it.

6.2 Visual Feedback

The visual feedback system was evaluated through a functional test designed to validate correct LED behavior in response to defined robot state inputs. The test objective was to confirm that specific robot states, when represented as a 6-bit binary input, resulted in the correct LED segments activating with the appropriate colors and patterns. Each bit represents a different robot state and should have the ability to be changed if the user inputs a new 6-bit string to the Pi terminal. The table below shows the different robot states used, with their respective color/pattern outputs, as well as the location on the arm base.

Table 12: Color/Pattern Outputs for Robot States

Robot Status	Color/Pattern Output	Location on Arm Base
Autonomous mode	Solid Purple	Front side
Tele-op mode	Solid Blue	Front side
Voice command active	Solid Green	Left side
Voice command recognized	Triple-flash Green (Only 1-2 seconds)	Left side
Voice command not recognized	Triple-flash Yellow (Only 1-2 seconds)	Left side
Ramp is detected	Solid Green	Right side
Cable was grasped	Solid Magenta	Right side
Emergency stop was engaged	Solid Red	Right side

Testing was conducted with the LED strips powered and controlled through the Raspberry Pi 4. The LED array was divided into predefined sections using Python arrays, with each section physically corresponding to a known location on the arm base. When looking at the robot front (Where the camera is installed), the user can identify three different sides of the arm base, which are the left, front, and right sides. Each side, with the aluminum angle installed, represents a robot's status, shown in the table above. Index 0 of the string input represents the robot's choice of mode, which can either be autonomous mode or tele-op mode. Index 1 is the voice command status, while index 2 is the voice command result. Index 3 and 4 are the detection status of the ramp and cables, respectively, and index 5 is the status of the emergency stops. The table below shows each of the robot states used, as well as the result if the user inputs 0 or 1 to the terminal.

Table 13: Robot State Input Bits

	Input Bit 0	Input Bit 1
Robot Functioning Mode	Autonomous Mode	Tele-Op Mode
Voice Command Status	Not Active	Active
Voice Command Result	Command not recognized	Command recognized
Ramp Detected	No ramp detected	Ramp detected
Cable Detected	No obstacle detected	Obstacle detected
Emergency Stop Status	Button not activated	Button activated

The binary input requires that the user input 0 or 1, or else the program will not successfully run and give an error message to the user. Each index is represented with a different status using the bit 0 or 1. For example, if the user were to input 011011 to the Pi terminal with this program, autonomous mode is set, voice command will be active, voice command will be recognized, no ramp is detected, cable is detected, and emergency stop is activated. Some of the inputs are dependent on one another, however. For example, if the voice command status is set to bit 0 (Not active), but the voice command result is set to bit 1 (command recognized), there will be no output for the voice command result despite this. This is trying to show that the voice command result should not output anything if the status isn't active. In addition, if the emergency stop status is activated, the output for this will override the outputs for the ramp and cable detections.

What this means is if the cable and ramp detections were set as active, but the emergency stop was active as well, then only the emergency stop output will show. Overall, the test results validate that the visual feedback system meets its functional requirements by accurately reflecting robot state information in real time. While full ROS2 integration was not completed within the semester, the successful standalone evaluation provides confidence that the subsystem logic is sound and can be reliably integrated into the broader system architecture in future work.

As for the integration, packages on the Pi 5 were downloaded successfully, meaning the visual feedback system can integrate with the system upgrade subsystem. The Pi 4 and 5 use the same GPIO logic, however, use different Neopixel packages for the LED strips. Each provide a 5 V power supply, which is what we need for this design.

6.3 Ramp Navigation

To test this subsystem, the ESP32 and compass was attached to a testing vehicle to simulate use on the robot. The compass was put through orientation testing such as a bearing check using angles marked on the ground to confirm the accuracy of the compass. Another test was performed to check if the compass could detect if it had surpassed 15 degrees in either pitch or roll. Then the correction code was tested by placing the compass on the ramp and simulating a path deviation to see if the proper consol outputs that will simulate corrective motor actions are outputted.

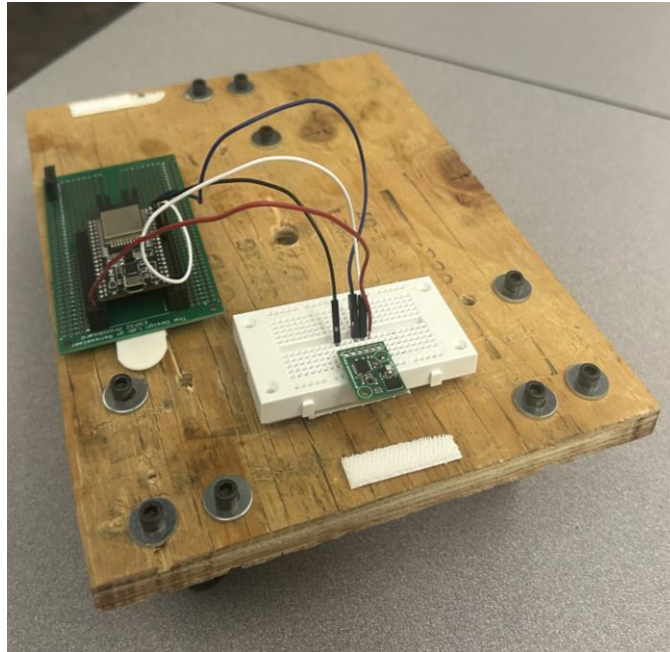


Figure 27: Compass Installation

6.4 Cable Detection

Unit testing for a perception-driven ROS 2 package must cover both pure software logic and the package's behavior as reflected in terminal outputs. The test strategy for this package is organized into three layers. First, function tests for image and geometry computations. Next, node-level tests using recorded data playback. Lastly, hardware validation tests that confirm performance on the physical sensor.

Functionality Tests

The core algorithm is decomposed into testable functions, each with clear inputs/outputs. These tests can be done without ROS running and without a camera connected. These tests include:

- Processing correctness: verify that resizing/filtering preserves image dimensions as expected and does not introduce invalid values.
- Cable mask/contour extraction: feed known test objects and verify that the detector returns a consistent set of cable pixels or a fitted curve/line.
- Depth sampling robustness: verify that invalid depth values are rejected and that sampling across cable pixels yields a stable aggregate depth when valid values are present.
- Pixel-to-3D projection: given known intrinsics and a set of coordinate values, verify that the computed (X, Y, Z) matches the expected results within numerical tolerance.
- No-detection logic: verify that the pipeline returns a clearly defined ‘no cable’ result instead of stale coordinates when the detector output is empty.

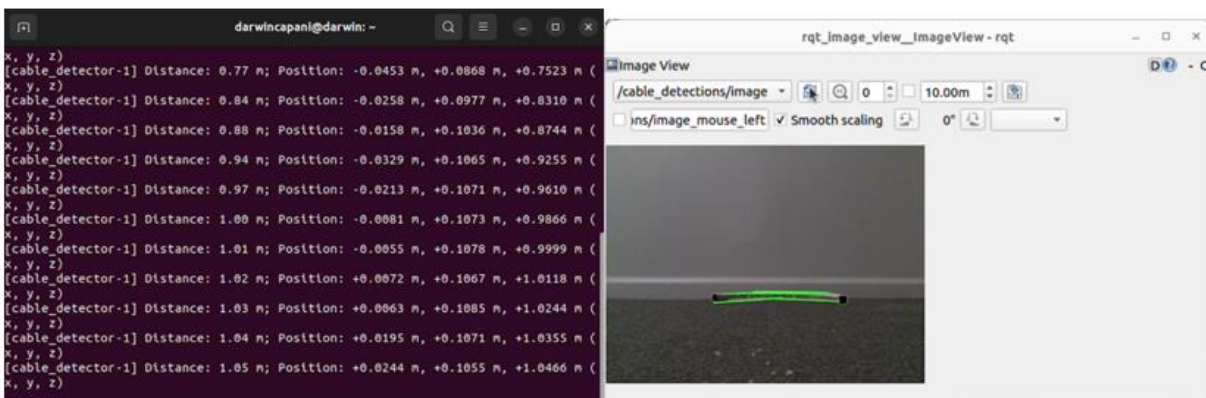


Figure 28: Cable detection functionality test results

Node Level Tests

Node-level tests make sure that the ROS 2 node correctly subscribes to the intended image topics, produces outputs in the expected formats, and maintains stable behavior over time. These tests are run by playing back color and depth and asserting properties of the published outputs.

- Topic contract tests: ensure the node subscribes to the configured color and aligned depth topics and publishes the overlay and coordinate outputs under the expected topic names.
- Output frequency and continuity: during playback, ensure outputs are produced for each valid input frame (or for each detection event) without unexpected stalls.
- Coordinate stability test: as the cable moves smoothly in the recorded sequence, verify that reported distance and XYZ values change smoothly without discontinuous jumps beyond a defined threshold (unless depth becomes invalid).
- No-cable segment test: on recorded segments where the cable is absent, verify that the node reports ‘no cable detected’ and does not publish misleading coordinates.

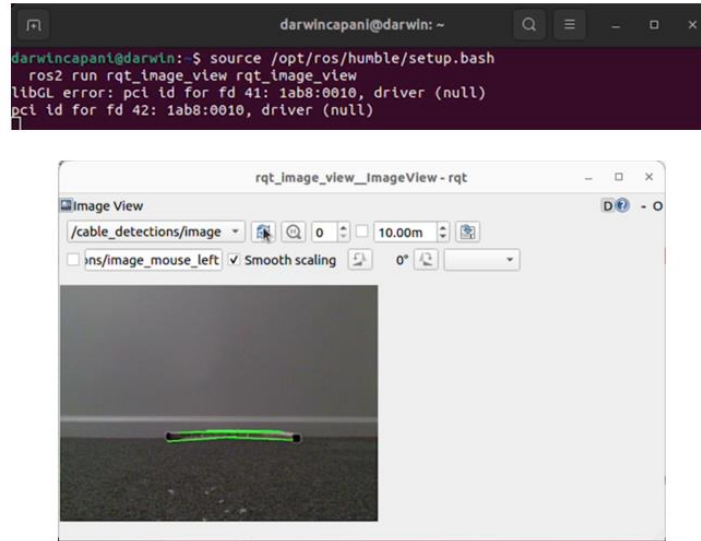


Figure 29: Cable detection node level test results

Hardware Tests

Because depth measurements depend on the physical sensor and environment, the final layer of testing is performed on the actual RealSense camera mounted on the robot (Figure 14).

Hardware-in-the-loop testing follows a controlled procedure:

1. Place a single cable at known distances and heights in front of the camera against a transparent background.
2. Confirm that the overlay line visually tracks the cable as it is moved laterally and in depth (Figure 16).
3. Record the reported Z (depth) and compare it against a tape measure or calibrated fixture to estimate absolute error.
4. Repeat at multiple distances to characterize sensor accuracy and identify depth-noise ranges.

5. Document results and analyze factors that may degrade performance, and ensure the node fails safely (no-detection) rather than outputting unstable coordinates.

6.5 Gripper Re-design

The Robotic Arm will be tested standalone or attached to the Robot base. 3 “cables” of diameters of 0.5, 1, and 2.5 in, with lengths of 4, 10, 15 in respectively will be placed in front of the robot. Each cable will be rotated clockwise or counterclockwise to simulate situations where the arm needs to adjust for different positions. The Robotic Arm will pick up each cable from the ground then lift the cable up to 10in and hold it there for 5 seconds. Then the arm will move back down to the ground and release the cable. This will be done for each of the 5 cables.

6.6 Pathfinding Algorithm

The path-finding subsystem’s unit test strategy (demo) will be the same as the integration for this work; the robot will be initialized into autonomous mode for it to detect obstacles and find a path from point A to point B without hitting any obstacles. The path-finding subsystem does not have many integrations; the most important one is to ensure that the code developed on python can be translated to the ROS2 and the raspberry pi and ensure that the robot is operational under autonomous mode. The unit test demo will show either a video of the robot functioning in autonomous mode: avoiding obstacles and moving, or a live demonstration that will demonstrate the same content as the video demonstration.

6.7 Safety Features

The Safety Features subsystem's unit test strategy (demo) will work in both standalone and integrated configurations. The subsystem consists of ultrasonic range sensors, and an emergency stop button, which together provide assistance in obstacle detection and immediate shutdown capability. The ultrasonic sensors will be mounted to a movable cart and tested under two scenarios: a clear path condition and a blind-spot condition, where a piece of cardboard is placed within the LiDAR blind spot area, approximately 6 inches from the robot's body.

During operation, the ultrasonic sensors will detect obstacles within the 'danger zone' of the robot to trigger a stop condition. This is displayed with a red LED that lights when something is within 6 inches of the robot chassis. The emergency stop button will be mounted to the rear of the robot and connected between the battery and the wheel motor controllers. When the emergency stop button is activated (pressed), it cuts all power to the wheels and halts their motion. An status LED on the motor controller turns off, showing that power has been removed from the system. Since the powerlines diverge between the rest of the robot and the motor controllers, the Raspberry Pi and subsequent electronics are still powered and operational.

Successful completion of this test is defined by reliable obstacle detection in blind-spot regions and immediate motor shutdown upon Emergency Stop activation.

7 Significant Accomplishments and Recommendations

7.1 System Upgrade

The main accomplishment is the creation of a docker image that pre-compiles all of the necessary dependencies for the robot. This allows for more efficient development for future teams as they will no longer be limited by the number of Pi boards, and allows for greater freedom in future hardware configurations. This accomplishment allows the work from previous

teams to continue to be usable while granting access to the increased computing power of the Pi

5. Currently, the docker image emulates the ROS2 Humble framework on an Ubuntu 22.04 setup, the same environment that was used in the original build and includes dependencies for the following packages that were part of the previous system:

- sllidar_ros2
- ds4_driver
- ds4_driver_msgs
- ds4drv
- base_control
- base_teleop

Dependencies were added for the following new packages:

- vision_control

For future teams, it is recommended to finish the installation of dependencies for the arm packages, which include `interbotix_ros_core`, `interbotix_ros_manipulators`, `moveit_visual_tools`, etc. Alongside that, the electrical system should be improved to support a more steady power supply to the Pi 5. This would reduce the risk of shutdowns from having low power.

Aside from that, future teams should explore the possibility of splitting the system into a lower powered computer on the robot that streams data from its sensors and a higher powered computing unit somewhere else on site that processes the data and sends back commands to the robot. This allows for lower power draw on the robot and truly remote operation, where the user won't need to see the robot and it's surroundings to properly pilot it.

7.2 Visual Feedback

The primary accomplishment of the visual feedback subsystem is the successful design and implementation of the LED interface that clearly communicates robot states to users and developers. A mechanically stable LED housing was designed using aluminum angles, ensuring consistent visibility while minimizing added weight. A scalable software architecture was developed to segment the LED strip into functional regions, enabling clear mapping between robot subsystems and visual outputs. The system was fully validated through functional testing, demonstrating correct behavior across multiple input conditions and enforcing appropriate safety-related dependencies between subsystems.

While this system achieved its core objectives, several open issues remain that should be accounted for in future work. Full integration of the LED subsystem into the ROS2 node graph has not yet been completed, limiting its ability to automatically reflect real-time system messages without manual input. Future teams should implement a dedicated ROS2 LED node that subscribes to standardized system state topics and publishes LED commands accordingly. The mechanical mounting solution was functional for testing but was not finalized for long-term deployment. Future work should replace the temporary mounting method of command strips with a permanent solution, such as custom brackets or industrial-grade adhesive systems, to improve durability during extended operation. Additionally, while the emergency stop LED rail has been identified in the design, it has not yet been validated against the physical E-Stop hardware. This integration should be completed to ensure immediate visual confirmation of emergency conditions.

7.3 Ramp Navigation

At the beginning of the semester, weight balancing was the focus. So, the first thing that occurred was an analysis of the existing CAD. Many of the models in the CAD did not have assigned material properties, which resulted in NX failing to analyze the weight distribution or returning an inaccurate value. Once the material properties were updated, it was found that the center of mass was nearly cantered with the robot, it was deemed that a dedicated counterweight would be unnecessary.

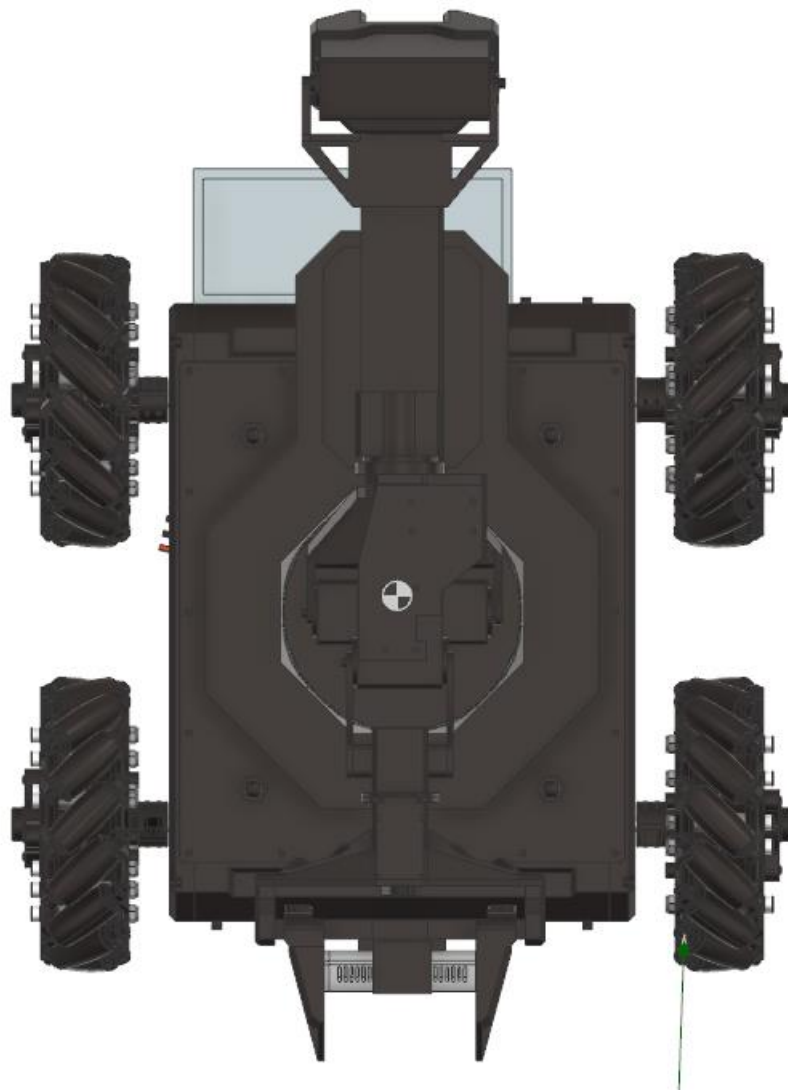


Figure 30: Robot COM top view

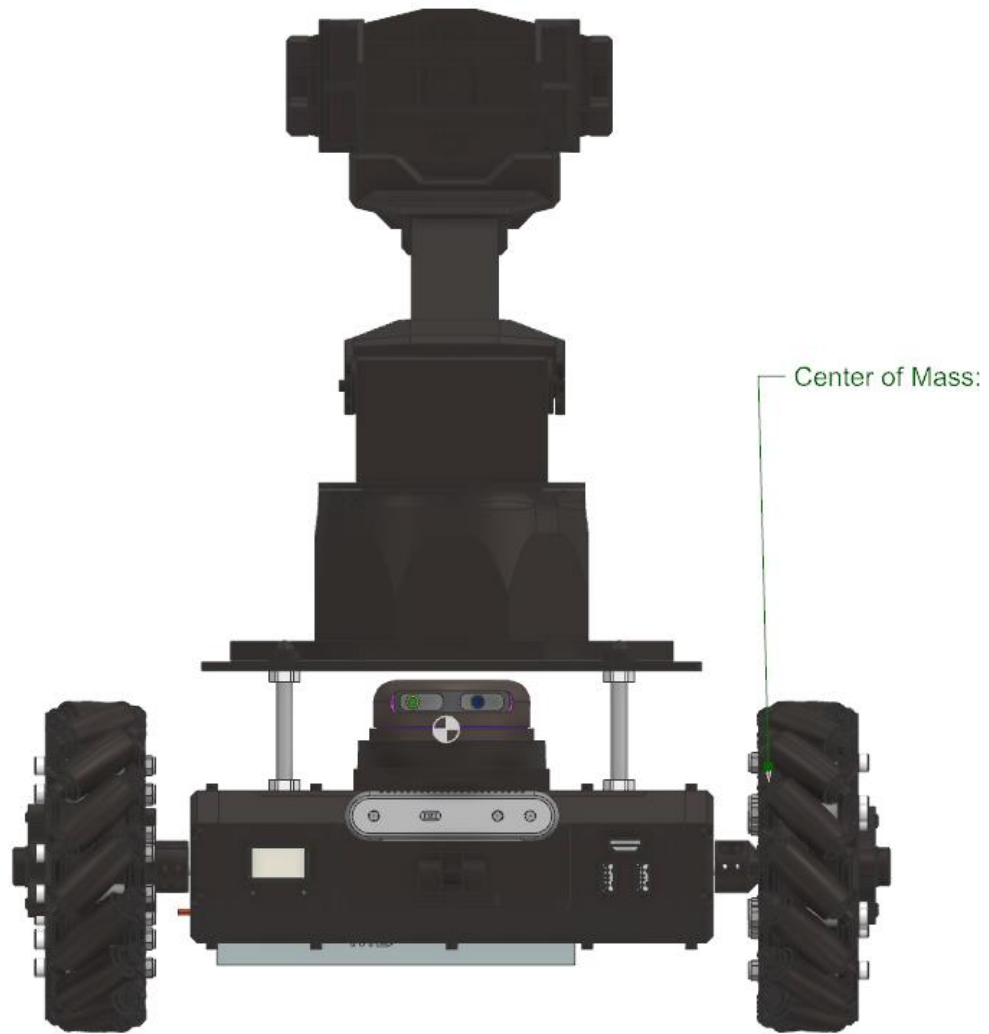


Figure 31: Robot COM front view

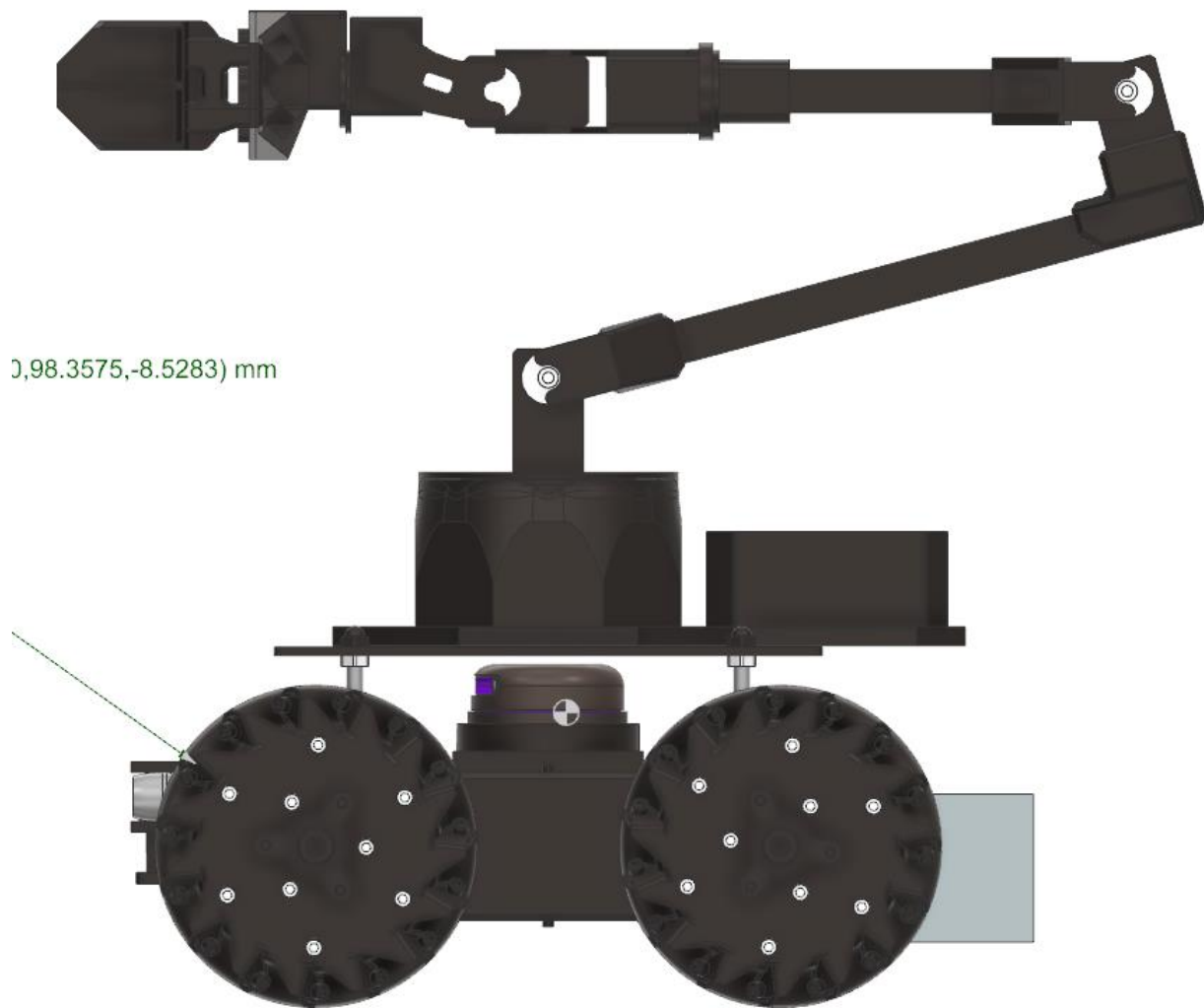


Figure 32: Robot COM side view

Moving on to the compass, it was decided that the the CMPS12 would be the best for the project's needs. The chip's delivery has been continually delayed, so the same model of the chip was borrowed from RPI resources. With the borrowed chip, compass initializing code as well as the beginning of ESP32 to PI 5 communication code has been made. The compass is within tolerance for its accuracy in its bearing, pitch, and roll measurements. Also, the external mount for the CMPS12 has been completed and attached to the underside of the robot.v

There are a few open issues that must be resolved before this subsystem can be complete. Firstly, it is unclear whether the hardware for the ESP32 to be integrated into the PI 5 will be completed before the semester ends. The compass has not been attached to the compass mount as it is not group property, so once the new compass arrives for the following semester's group it may be attached and integrated. The motor control code needs to be edited to take into account the robot heading and correct when necessary.

7.4 Cable Detection

A ROS 2 cable detection and localization package was built and integrated to give real-time feedback on where a cable is in front of the robot. In the final setup, an Intel RealSense RGB-D camera is mounted on the front side of the robot base, facing outward toward the workspace. The package detects the cable in the RGB image, draws a green line overlay that tracks the cable as it moves, and converts the detected cable pixels into distance and XYZ coordinates using the aligned depth-to-color stream. The system runs continuously and publishes ROS 2 outputs, which are the overlay image and 3D point data, making it usable both for demonstrations and for downstream autonomy modules that need cable location data.

For recommendations, the next priorities are improving the measurement of confidence and robustness. The first step is to add a simple accuracy test procedure with known distances or offsets, so the XYZ output can be reported with clear error bounds. The second is to reduce depth noise effects by adding light filtering, like smoothing or rejecting obvious outliers, to make the coordinates less jittery. Finally, the detector should be stress-tested in more realistic scenes, like different lighting or floor textures, to tune thresholds and confirm the overlay and XYZ output stay stable outside of a clear background.

7.5 Gripper Re-design

A fractal vise inspired gripper design was 3D printed and adapted for integration with the robotic arm following significant revisions to the original gripper configuration. Testing conducted during the previous semester showed that the original design could reliably handle smaller diameter cables but consistently struggled to grip larger cables. To address this limitation, the fractal vise inspired geometry was selected for its ability to increase the surface area in contact with the cable, thereby improving overall gripping performance. The new design was integrated into the existing stock gripper structure to maintain compatibility with the robotic arm while simplifying the design and 3D printing process. Through three iterations, each improving on the previous version by increasing jaw height, improving printing tolerances, and increasing the gap between the jaws to accommodate larger cables. And as a result the gripper demonstrated improved reliability and successfully gripped cables with outer diameters ranging from 0.5 to 2.5 inches.

7.6 Pathfinding Algorithm

For the pathfinding subsystem, the LiDAR can be used to detect obstacles and send its data into another node for path planning. The A* search algorithm works perfectly and is able to generate a path from a starting point to an end point almost instantly. The path-finding algorithm designed will be able to move from point to point completely autonomously when there are no obstacles blocking its path and the robot will also stop all movements when it has determined that there is no viable path available. However, the robot was unable to accurately judge the path when there is obstacle present and will sometimes run into them. The path-finding algorithm did somewhat work before Thanksgiving; however, the code broke after the team returned from the

break and was unable to recover the working version. The recovered program, however, is still kind of faulty in its execution, the robot when in autonomous mode will begin to move out of the way of an obstacle but will resume back to its initial direction thus hitting the obstacle.

Increasing the frequency of the path replan has already been implemented but was unfortunately unable to be tested and demonstrated due to issues with the Pi-4. Future recommendations is to fully debug the autonomous mode operations and develop a return to home feature that will have the robot autonomously return back to its starting point after the robot reached an end goal.

7.7 Safety Features

The Safety Features subsystem was successfully designed, implemented, and independently validated to support safe operation. The subsystem consists of a normally closed (NC) Emergency Stop (E-Stop) button and ultrasonic range sensors, each addressing safety from different perspectives. While full system integration with the Raspberry Pi was not completed, both components were fully functional and tested in isolation, meeting their intended objectives.

The E-Stop mechanism reliably disconnected power to the motor controllers upon activation, providing immediate physical shutdown of the wheel motors. This ensured it would work as a fail-safe, as it was independent of software execution or processor status. Ultrasonic sensors were configured and tested to detect obstacles within a defined short-range threshold, demonstrating consistent and repeatable distance measurements.

Integration with the Raspberry Pi was not performed due to limited system access during testing, as the Pi was allocated to higher-priority, computation intensive, subsystems requiring direct hardware interaction. The Safety Features subsystem was validated as a standalone module

because of this, allowing focused debugging, verification, and testing without interference, or interfering, the other subsystems.

Future work should prioritize establishing a communication interface between the Safety Features subsystem and the Raspberry Pi to enable software level awareness of the E-Stop's activation status and obstacle detection status. This would allow for coordinated responses with the Visual Feedback subsystem, as well as others, to alert operators when the robot is stuck or in an Emergency Stop state, whether they are at a monitor away from the robot or the one pressing the button. This can be done with the current Arduino, where the pins can be connected to the other leads on the E-Stop, and the Raspberry Pi can be connected to the Arduino. Despite the lack of full integration, the subsystem demonstrated reliable independent operation and is well positioned for the future semester's work on finishing integration.

8 Conclusion

The Fall 2025 Aircraft Robotic Assembly team successfully advanced the Boeing-sponsored robotic assistant by addressing several critical limitations identified in prior semesters and laying a stronger foundation for future development. This work focused on improving system usability, safety, perception, and computational capability to better support autonomous cable-handling operations in a simulated aircraft manufacturing environment.

Key outcomes of this semester include the successful upgrade to a Raspberry Pi 5 platform, enabling improved computational performance and future scalability, and the development of a visual feedback subsystem that provides a clear, immediate, and intuitive indication of robot states. The visual feedback system enhances both debugging efficiency and operator awareness, directly supporting safer human-robot interaction. Significant progress was also made in cable detection through the integration of an RGB-D perception pipeline capable of identifying cables and estimating their 3D position in real time, producing outputs suitable for downstream autonomy and manipulation tasks. Additionally, a redesigned fractal-inspired gripper demonstrated improved reliability when handling cables of varying diameters, addressing one of the most persistent mechanical challenges from previous semesters.

Safety was further strengthened through the implementation of a hardware-level emergency stop and ultrasonic sensors to cover LiDAR blind spots. While full software integration of these safety features was not completed, the subsystem was validated independently and provides a robust, fail-safe mechanism that can be integrated with higher-level control and feedback systems in future work. Progress was also made toward improved navigation and ramp handling through compass-based orientation sensing, establishing a pathway for drift correction and more reliable traversal of inclined surfaces.

The results of this work will be used by future capstone teams to complete system-level integration, refine autonomous path planning, and fully unify perception, navigation, safety, and feedback subsystems under ROS2. Boeing engineers and technicians stand to benefit from a more reliable, intuitive, and safer robotic assistant that reduces physical strain, improves efficiency, and supports consistent cable installation in complex assembly environments. Ultimately, this semester's contributions move the project closer to a deployable, industrially relevant robotic system capable of collaborative operation on aircraft factory floors.

9 References

- [1] R. D. Lab, *Final Design Report – S25*, RPI Capstone Design Team, Troy, NY, USA, 2025.
- [2] Open Robotics, “ROS 2 Humble documentation,” *ros.org*. [Online]. Available: <https://docs.ros.org/en/humble/index.html>. Accessed: Oct. 16, 2025.
- [3] A. R. A. F. 2 Team, “Designlab@RPI: Boeing robotic assembly software repository,” Design Lab, Rensselaer Polytechnic Institute, Jul. 2024. [Online]. Available: https://designlab.eng.rpi.edu/edn/projects/boeing-robo-assy/repository/495/entry/tags/S24/Software/Docker_test_routine.md. Accessed: Dec. 2025.
- [4] A. R. A. S. Team, *Final Design Report S25*, Design Lab, Rensselaer Polytechnic Institute, Apr. 2025. [Online]. Available: <https://designlab.eng.rpi.edu/edn/projects/boeing-robo-assy/repository/495/entry/tags/S25/Reports/Final%20Deliverables/Final%20Design%20Report%20S25.docx>. Accessed: Dec. 2025.
- [5] IBM Research Division, *An Updated Performance Comparison of Virtual Machines*, IBM, 2014.
- [6] McMaster-Carr, “Aluminum angles.” [Online]. Available: <https://www.mcmaster.com/8982K523/>. Accessed: Dec. 2025.
- [7] R. Santos, “Raspberry Pi GPIO pinout,” *Random Nerd Tutorials*. [Online]. Available: <https://randomnerdtutorials.com/raspberry-pi-pinout-gpios/>. Accessed: Dec. 2025.
- [8] All3DP, “Fractal vise: 3D-printed tool.” [Online]. Available: <https://all3dp.com/1/3d-print-tools-fractal-vise/>. Accessed: Dec. 2025.
- [9] Arduino, “Getting started with the Arduino Nano.” [Online]. Available: <https://docs.arduino.cc/tutorials/nano/nano-getting-started/>. Accessed: Dec. 2025.
- [10] SparkFun Electronics, *HC-SR04 Ultrasonic Distance Sensor Datasheet*. [Online]. Available: <https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>. Accessed: Dec. 2025.
- [11] Instructables, “Emergency stop button.” [Online]. Available: <https://www.instructables.com/Emergency-Stop-Button/>. Accessed: Dec. 2025.
- [12] Intel RealSense, “Compiling librealsense for Linux Ubuntu.” [Online]. Available: <https://dev.realsenseai.com/docs/compiling-librealsense-for-linux-ubuntu-guide>. Accessed: Dec. 2025.
- [13] Intel RealSense, “RealSense ROS wrapper.” [Online]. Available: <https://github.com/realsenseai/realsense-ros>. Accessed: Dec. 2025.

- [14] Open Robotics, “Writing a simple publisher and subscriber (Python),” *ROS 2 Foxy Tutorials*. [Online]. Available: <https://docs.ros.org/en/foxy/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Publisher-And-Subscriber.html>. Accessed: Dec. 2025.
- [15] Open Robotics, “cv_bridge package overview.” [Online]. Available: https://index.ros.org/p/cv_bridge/. Accessed: Dec. 2025.
- [16] Open Robotics, “Image_transport tutorials.” [Online]. Available: https://wiki.ros.org/image_transport/Tutorials/SubscribingToImages. Accessed: Dec. 2025.
- [17] ROBOTIS, “XM series servo motor specifications.” [Online]. Available: <https://www.robotis.us/xm/>. Accessed: Dec. 2025.
- [18] J. A. Castro, “CMPS12-lib-Circuitpython,” *GitHub repository*, 2025. [Online]. Available: <https://github.com/JuliansCastro/CMPS12-lib-Circuitpython>. [Accessed: Dec. 2025].

Appendix A: Standards

Table 14: Standards Table

Category	Standards	Purpose / Application
Software	ROS2	Framework that allows for easy compartmentalization of software into “nodes”, usually meant for developing robots
	Ubuntu 22.04 LTS	Operating System to control the Pi
	Python3	Coding language used to program the robot
	Bluetooth	Connects to headset for voice commands
	PEP8 – Python Style Guide for Code Readability	For a ROS2 Python node that controls the WS2811 LED Strip
	ISO 9241-210:2019 – Ergonomics of Human–System Interaction	Provides guides for designing user-centered interfaces
	ISO 3691-4 (Safety of Automated Guided Vehicles - AGVs)	Defines requirements for autonomous vehicle navigation and obstacle avoidance
	IEC 61508: Functional safety of electrical/electronic/programmable electronic safety-related systems	Promotes fault-tolerant software designs in ROS/Linux setups to handle failures gracefully and enable safe return-home initiation.
	ISO/IEC/IEEE 29119 – Software Testing Standards	Provides a framework for testing and validating safety-critical software.
Electrical	5V/3A Power	Required to power the Raspberry Pi
	IEC 60204-1:2016 – Safety of Machinery: Electrical Equipment	Establishes safe wiring and electrical protection requirements for connecting the LED strip
	ISO 13849-1 (Safety of Control Systems in Machinery)	The robot has safety-related electrical control systems for obstacle avoidance, ensuring proper fail-safe and risk mitigation.
	Manufacturing ISO 9001	Defines the outlines for using Raspberry Pi in a production environment
	USB 3.0 Standard	Governs the data communication protocol between the RealSense camera and the Jetson Nano, ensuring reliable, high-speed data transfer.
	IEEE 802.3 (Ethernet)	If the computer communicates over wired Ethernet, this standard ensures reliable network communication with other components.

Mechanical	IEC 60529 – Degrees of Protection	Ensures protection of the LED strip from dust or debris
	ANSI B.11 series	Safety protocols for working with CNC equipment for creating counterweight.
	ISO Metric Screw Thread Standards	Ensures that all mounting hardware (bolts, screws) used to secure the camera and Jetson Nano are standardized and compatible.
Industrial	ANSI/RIA R15.06 – Industrial Robot Safety Standard	Defines safety and communication guidelines for human interaction with robots
	ISO 13850 – Safety of Machinery: Emergency Stop Function	Defines design and performance criteria for emergency stop systems to ensure rapid, reliable shutdown.
	ISO 10218 - Robots and robotic devices	Provides safety guidelines for industrial robots. While the rover is not a traditional industrial arm, these principles inform safe design.

Appendix B: Bill of Materials

Main BOM

Table 15: Main BOM

Part	Part Number	Price	Quantity	Total
Lynxmotion - A4WD3 Rugged Mecanum Rover Autonomous Kit	RB-Lyn-1135	\$895.95	1	\$895.95
ViperX 300 Robot Arm 6DOF	KIT-VIPX300-6DOF	\$6,129.95	1	\$6,129.95
Raspberry Pi 4B	B07TD42S27	\$52.10	1	\$52.10
Waveshare AC8265 Wireless NIC Module	AC8265	\$21.99	1	\$21.99
RPLidar A2M12 360 Laser Range Scanner	A2M12	\$229	1	\$229
33k Ohm Resistor	E2433.0	\$0.10	2	\$0.20
10k Ohm Resistor	E241.0	\$0.10	1	\$0.10
Male USB Breakout	B09LC8WQCD	\$0.65	10	\$6.49
Female USB Breakout	B07W7XWV3W	\$0.70	10	\$6.99
USB Hub	B088KBGRD9	\$7.99	1	\$7.99
HDMI Ribbon Cable	B09F61FZWQ	\$8.99	1	\$8.99
90-Degree Micro USB Cable	B0CBG384N6	\$4.50	2	\$8.99
OLED Display	B09C5K91H7	\$3.00	5	\$14.99
Power Module (XL4015)	DFR0205	\$8.50	1	\$8.50
DaierTek 12V Switch	B07T6XWZKN	\$10.89	1 (2)	\$10.89
PS4 Controller	CUH-ZCT2U	\$64.99	1	\$64.99
Ring Terminal Pack	WM13633-ND	\$7.95	1	\$7.95
25A Blade Fuse Pack	BP/ATC-25-RP	\$4.75	1	\$4.75
Ordentlich Latching Push Button	B0CQ7PTF9K	\$9.99	1	\$9.99
KBT Lithium-ion Batteries	B0C23Y3VZK	\$32.99	2	\$65.98
Yipin Hexha 12V/24V to 5V Voltage Regulator/Converter	12V/24VTO5V20A	\$18.99	1	\$18.99
USB C to Bare Wire Cable	B0CCYVZ52R	\$8.99	1	\$8.99
Female to Dual Male XT60 Y Splitter	B0C4PG2J25	\$12.99	1	\$12.99
SanDisk MAX Endurance 32GB TF Card	B08HVXVWPN	\$33.46	1	\$33.46
NFHK CYFPV Up Angled 90 Degree HDMI Male to Female FPC Flat Cable	B09F61FZWQ	\$8.99	2	\$17.98

			Total:	\$7,649.19
--	--	--	---------------	-------------------

Fall 2025 BOM

Table 16: Fall 2025 BOM

Part	Part Number	Price	Quantity	Total
Compass	RB-Lyn-1135	\$129.99	1	\$129.99
WS2811 LED Strip	B01CNL6K52	\$17.33	1	\$17.33
Aluminum 90 Degree Angle	8982K523	\$7.01	1	\$7.01
Fibreheart Rebound Flexible Filament	PEBA 95A	\$54.99	1	\$54.99
			Total:	\$209.32

Appendix C: Ethical and Professional Responsibilities

Table 17: Team Ethical and Professional Responsibilities

Issues (Context)	Impact, 1 (Low) – 5 (High)	Description of Impact and Related Project Decisions
Global	4	Computers are a multinational product. The choice of computer to use on a robot with a crucial role in the assembly of aircraft, an industry closely associated with sensitive industries like the defense industry, means that a lot of consideration needs to be put in to ensure that the supplier of the chip will be able to provide a steady supply and remain trustworthy enough to continually supply those components. In that regard, the Raspberry Pi Foundation is based in the UK, a close ally.
	3	The WS2812 LED visual feedback system contributes to safer and more interpretable human-robot interaction, which is a universal requirement in manufacturing environments around the world. Because the subsystem uses inexpensive, openly available hardware, it can be replicated globally without supply issues. Some differences exist in robotic safety standards, which include varying color-coding, which is why we ensure that the visual states follow widely recognized industrial color conventions.
	3	Boeing is an international company, so the performance of this robot, even down to its ramp climbing capabilities, will be viewed by many Boeing employees around the world. The overall effectiveness of the robot directly reflects Boeing's quality and trustworthiness, so it is imperative that it be as robust as possible in all its subsystems.
	4	Cable detection can be deployed in labs, hospitals, warehouses, and public-facing environments across many countries where safety expectations are high. If the system produces incorrect detections, a robot may collide with equipment or snag a cable, creating risk of injury and property damage. Because the output includes 3D XYZ coordinates, inaccurate depth association can cause unsafe navigation decisions in any region. To address this, the package was designed around aligned depth-to-color so the depth sample corresponds to the same moment and pixel location as the detected cable. The system also provides visual confirmation through green-line overlays so operators can verify what the robot is interpreting. The node's outputs are exposed through standard ROS2 topics to support integration reviews and independent validation by other teams. The README documents operating assumptions and limitations so

Issues (Context)	Impact, 1 (Low) – 5 (High)	Description of Impact and Related Project Decisions
		users do not over-trust the system in conditions where performance degrades.
	3	The Fractal Gripper can be used in various applications outside of gripping cables. The unique ability of the jaws to conform to objects placed between them allows for the use in any application where awkward or uncommon geometry needs to be secured. The high repeatability of the system builds trust in autonomous systems, especially when those systems work closely with other humans.
	3	The path-finding algorithm used by the aircraft robotic assembly project's robot in its autonomous mode contributes to greater safety during interactions between individuals and robots. By accurately navigating the workplace and avoiding obstacles, the algorithm lowers the chance of accidents and can create a culture of trust and safety within organizations that deploy autonomous robots. This culture of better safety protocols could build trust in the workers and lead to closer integration of robotic technologies into everyday workplace.
	5	The safety features implemented in the Aircraft Robotic Assembly project directly contribute to protecting individuals who interact with robots in shared workspaces. By incorporating a hardware level Emergency Stop and short range obstacle detection, the system lines up with widely accepted industrial safety practices for robotic platforms. These features help reduce the likelihood of unsafe operation.
Economic	4	Due to the current demand for computer components like RAM and storage, it is incredibly expensive to source computer components at the moment. With the new docker setup on the Pi 5, the range of available components to choose from grows exponentially, reducing some of those concerns. The widespread compatibility also allows for saved labor costs by lessening the need to constantly overhaul the system every time a change needs to be made to the hardware.
	3	The subsystem improves testing efficiency and reduces debugging time by giving engineers immediate state awareness, lowering labor costs during development and maintenance. WS2812 strips also provide a cheaper alternative to industrial tower lights or commercial indicator systems. However, integrating the LEDs requires additional wiring, power routing, and potential replacement costs if sections fail. To keep the subsystem economically feasible, we powered the LEDs with

Issues (Context)	Impact, 1 (Low) – 5 (High)	Description of Impact and Related Project Decisions
		the existing Raspberry Pi 4/5 instead of adding a new power source to use.
	1	The ability of the robot to climb ramps has very little effect on its worth and int price of construction. There is only one sensor of difference between a Aircraft Robotic Assembly Robot that can climb a ramp and one that cannot.
	3	Cables are common in workspaces, and collisions can create downtime, repairs, and replacement costs for both cables and connected devices. A reliable detector can reduce these costs by prompting safer motion decisions before contact occurs. An unreliable detector can also increase costs through false positives that trigger unnecessary stops and reduce productivity. The project used a RealSense RGB-D camera and ROS2 interfaces to keep the solution accessible and affordable for student and lab environments. The node was structured to run continuously without queue buildup so latency does not cause delayed or incorrect actions that waste time. The package publishes distance and XYZ outputs so higher-level autonomy can make consistent decisions without manual measurement. The installation instructions were updated to follow the official librealsense workflow, so setup time and support burden are reduced.
	3	The 3D printable nature of the gripper increases cost savings as compared to injection molding or CNC machining, 3D printing is faster and cheaper to produce. Custom molds do not need to be made for injection molding, and CNC machines, engineers, and programs do not need to be tested, reducing production time and costs. In addition, reduced failures from drops or items slipping out from the jaws, also aid in reduced overall costs.
	3	The implementation of a reliable path-finding algorithm helps in avoiding obstacles, which prevents damage to both the robot and whatever the robot could potentially crash into. Collision prevention indirectly contributes to cost saving, as it minimizes repair and replacement costs. Collision prevention also provides economically in another way, as repairs or replacement could potentially take a long time, collision prevention can sustain operation without any unplanned downtime due to accidents, thus contributing to a workplace's overall efficiency.
	4	The implementation of reliable safety mechanisms helps prevent damage to the robot and surrounding equipment by enabling immediate shutdown during emergency conditions. Avoiding collisions and uncontrolled movement reduces repair,

Issues (Context)	Impact, 1 (Low) – 5 (High)	Description of Impact and Related Project Decisions
		replacement, and maintenance costs, while also minimizing unplanned downtime. Using low-cost, readily available components like the HC-SR04 ultrasonic sensors allows the subsystem to achieve these safety benefits without minimal cost.
Environmental	4	Power draw is a large concern when working with electronics in an era where climate change is a growing concern and the energy industry struggles to convert to green sources at the rate at which our energy consumption grows. This is why it is essential that the chosen processor be power efficient, especially when the robot does not need all the power that the chip can provide at once.
	4	WS2812 strips contain embedded micro-LEDs that are not repairable, which contribute to long-term waste. The system also draws continuous electrical power during operation. To minimize the environmental impact, we switched LED modules to use a 5V power instead of 12V to lower energy consumption and decrease the risk of needing replacements for the burnt-out LED rails.
	2	The robot uses electricity, is comprised of electronics made from rare metals, and is used in the construction of planes. The robot is not actively polluting or depolluting the environment whilst doing its job, though it does still consume electricity and take some rare materials to produce.
	2	The software's environmental impact is indirect, but it can reduce waste by preventing cable damage and preventing damage to equipment that would otherwise be discarded. If a robot repeatedly snags cables, the resulting replacements and repairs increase material use and e-waste. The package was designed to process data efficiently in real time so it does not require excessive compute power or energy. The system is intended to run locally on the robot and does not require cloud processing, which reduces network and storage overhead. The green-line overlay helps teams debug quickly, which reduces repeated trial-and-error testing that can lead to unnecessary hardware changes. The README encourages using existing compatible hardware when possible rather than purchasing new components without need. Overall, the project decisions prioritize robustness and efficient operation to minimize unnecessary physical damage and resource consumption.
	2	PLA is a recyclable and plant-based material. It is easy to shred down used or broken gripper parts and reform them into

Issues (Context)	Impact, 1 (Low) – 5 (High)	Description of Impact and Related Project Decisions
		filaments to be used to print newer ones. However, although PLA is plant based, it requires specific conditions to breakdown, but companies specializing in PLA and 3D print waste composting exist.
	2	By preventing collisions through implementing a reliable path-finding algorithm, one could reduce the risk of robot related waste, as collisions could sometimes render tools or materials inoperable. Utilizing a reliable path-finding algorithm supports sustainable practices by utilizing resources more efficiently, ultimately reducing the environmental footprint that enables collision prevention.
	2	By preventing collisions and unintended motion, the safety subsystem reduces the risk of damage to tools, cables, and robotic components that could otherwise become waste. While the ultrasonic sensors and supporting electronics require continuous low-level power during operation, their energy consumption is minimal relative to the full system, taking only ~5V. This contributes indirectly to sustainability by extending equipment lifespan and helping reduce material waste.
Societal	1	The choice of system has little bearing on society, as the effect is entirely from the software programmed onto the system, not the system itself. No computer is inherently more safe or disability-accommodating than any other.
	3	Clear visual feedback increases safety for users by showing robot states in real time. This reduces the risk of accidental collisions and helps workers quickly understand whether the robot is safe to approach. In a factory setting, clearly showing states such as autonomous or tele-op control, emergency stop, or cable handling helps prevent misunderstandings that could lead to unsafe interactions. However, reliance on color-coded signals may disadvantage color-blind or visually impaired users. To address this, the visual feedback system will be paired with complementary feedback (Text on the Pi terminal), to allow for more than one form of communication.
	1	The robot has very limited societal impact. Its purpose is to aid in the construction of planes. Discussion about the robot should not extend past its purpose in the workplace or ideas about improving its performance.
	5	Camera-based perception in shared spaces can affect human safety, trust, and comfort around robots. If cable detection fails, the robot may pull a cable, create a trip hazard, or cause connected equipment to fall, which can harm people nearby.

Issues (Context)	Impact, 1 (Low) – 5 (High)	Description of Impact and Related Project Decisions
		Because the package outputs motion-relevant XYZ coordinates, errors can propagate into navigation behaviors with real safety consequences. The project emphasizes transparency by drawing the detected cable as green lines, which allows immediate human verification of what the system believes is a cable. The node also publishes a simple detected state and distance so higher-level control can choose conservative behaviors such as slowing down or stopping. The design supports privacy-aware operation because it focuses on geometric cable detection and does not require storing video data for normal use. The documentation states limits and failure cases so users understand when supervision is needed and when the system should not be relied on alone.
	2	An environment where an operator can trust that any object that the robot picks up will be secure is one that builds trust and could increase productivity.
	3	Collision prevention and its overall autonomous goal ensure lowering of risks of physical injuries to workers during its operation. An environment where workers feel safe from potential injuries could foster more productivity and higher morale. This also enhances trust amongst the workers who work alongside robots, which can stimulate innovation and promote a more agile workforce that is open to adapt to future technologies.
	4	Emergency shutdown capability and obstacle detection reduce the risk of physical injury to operators and bystanders during robot operation. The presence of a physical Emergency Stop button increases impact, as it allows the user to have an immediate method for halting movement of the robots wheels, regardless of software state.